

SOLIDWORKS API (.NET VBA C#) — Examples (6/50):

Change_Angle_to_Supplementary_Angle_Example_VBNET

Combine_Bodies_Example_VBNET

SOLIDWORKS API Documentation (converted from CHM)

Change Angle to Supplementary Angle Example (C#)

This example shows how to change the angle in an angular dimension to its supplementary angle.

```
//-----
// Preconditions: Verify that the drawing exists.
//
// Postconditions:
// 1. Opens the drawing.
// 2. Selects the angular dimension.
// 3. Changes the angle to its supplementary angle.
// 4. Examine the graphics area.
//
// NOTE: Because the drawing is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            bool status = false;
            int warnings = 0;
            int errors = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\clamp1.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select angular dimension
            status = swModelDocExt.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.115166498498499, 0.167429477477477, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swDisplayDimension = (DisplayDimension)swSelectionMgr.GetSelectedObject6(1, -1);

            //Change angle to supplementary angle
            status = swDisplayDimension.SupplementaryAngle();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Change Angle to Supplementary Angle Example (VBA)

This example shows how to change the angle in an angular dimension to its supplementary angle.

```
'-----  
' Preconditions: Verify that the drawing exists.  
'  
' Postconditions:  
' 1. Opens the drawing.  
' 2. Selects the angular dimension.  
' 3. Changes the angle to its supplementary angle.  
' 4. Examine the graphics area.  
'  
' NOTE: Because the drawing is used elsewhere, do not save  
' changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swDisplayDimension As SldWorks.DisplayDimension  
Dim status As Boolean  
Dim warnings As Long  
Dim errors As Long  
Dim fileName As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.slddrw"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
  
    'Select angular dimension  
    status = swModelDocExt.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.115166498498499, 0.167429477477477, 0, False, 0, Nothing, 0)  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swDisplayDimension = swSelectionMgr.GetSelectedObject6(1, -1)  
  
    'Change angle to supplementary angle  
    status = swDisplayDimension.SupplementaryAngle  
  
End Sub
```

Change Angle to Supplementary Angle Example (VB.NET)

This example shows how to change the angle in an angular dimension to its supplementary angle.

```

-----
' Preconditions: Verify that the drawing exists.
-----
' Postconditions:
' 1. Opens the drawing.
' 2. Selects the angular dimension.
' 3. Changes the angle to its supplementary angle.
' 4. Examine the graphics area.
' NOTE: Because the drawing is used elsewhere, do not save
' changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim status As Boolean
        Dim warnings As Integer
        Dim errors As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Select angular dimension
        status = swModelDocExt.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.115166498498499, 0.167429477477477, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swDisplayDimension = swSelectionMgr.GetSelectedObject6(1, -1)

        'Change angle to supplementary angle
        status = swDisplayDimension.SupplementaryAngle

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Change Bend Radius of Sheet Metal Part Example (VBA)

This example shows how to change the default bend radius of a sheet metal part.

```
'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Select the Sheet-Metal feature.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Doubles the default bend radius .
' 2. Examine the graphics area and Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSheetMetal As SldWorks.SheetMetalFeatureData
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swSheetMetal = swFeat.GetDefinition

    Debug.Print "Feature = " & swFeat.Name
    Debug.Print " Original bend radius = " & swSheetMetal.BendRadius * 1000# & " mm"

    ' Rollback to change default bend radius
    bRet = swSheetMetal.AccessSelections(swModel, Nothing): Debug.Assert bRet

    ' Double the default bend radius value
    swSheetMetal.BendRadius = 2# * swSheetMetal.BendRadius

    ' Apply changes
    bRet = swFeat.ModifyDefinition(swSheetMetal, swModel, Nothing): Debug.Assert bRet

    Debug.Print " Modified bend radius = " & swSheetMetal.BendRadius * 1000# & " mm"

End Sub
```

Change Bend Radius of Sketched Bend Example VB (Topic)

SOLIDWORKS API Help

Change Bend Radius of Sketched Bend Example (VBA)

This example shows how to change the bend radius of a sketched bend.

```
.....
' Preconditions:
' 1. Open a model document containing a sketched bend.
' 2. Sketched bend is selected.
'
' Postconditions: Increases the size of bend radius
' of the selected sketched bend is increased by 1.5 times.
.....

Option Explicit

Public Enum swFlangePositionTypes_e
    swFlangePositionTypeMaterialInside = 1
    swFlangePositionTypeMaterialOutside = 2
    swFlangePositionTypeBendOutside = 3
    swFlangePositionTypeBendCenterLine = 4
    swFlangePositionTypeBendSharp = 5
End Enum

Public Enum swBendAllowanceTypes_e
    swBendAllowanceBendTable = 1
    swBendAllowanceKFactor = 2
    swBendAllowanceDirect = 3
    swBendAllowanceDeduction = 4
End Enum

Sub DumpCustomBendAllowanceInfo _
(
    sPadStr As String, _
    swCustBendAllow As SldWorks.CustomBendAllowance _
)
    Debug.Print sPadStr & "Type" = " & swCustBendAllow.Type
    Debug.Print sPadStr & "BendAllowance" = " & swCustBendAllow.BendAllowance * 1000# & " mm"
    Debug.Print sPadStr & "BendDeduction" = " & swCustBendAllow.BendDeduction * 1000# & " mm"
    Debug.Print sPadStr & "BendTableFile" = " & swCustBendAllow.BendTableFile
    Debug.Print sPadStr & "KFactor" = " & swCustBendAllow.KFactor
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swFeat As SldWorks.feature
    Dim swSketchBend As SldWorks.SketchedBendFeatureData
    Dim swBendFeat As SldWorks.BendsFeatureData
    Dim swCustBendAllow As SldWorks.CustomBendAllowance
    Dim swFace As SldWorks.face2
    Dim swEnt As SldWorks.Entity
    Dim nFace_X As Double
    Dim nFace_Y As Double
    Dim nFace_Z As Double
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swSketchBend = swFeat.GetDefinition
    Set swCustBendAllow = swSketchBend.GetCustomBendAllowance

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name
    '1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Debug.Print " BendAngle" = " & swSketchBend.BendAngle * 57.3 & " deg"
    Debug.Print " BendRadius" = " & swSketchBend.BendRadius * 1000# & " mm"
    Debug.Print " PositionType" = " & swSketchBend.PositionType
    Debug.Print " ReverseDirection" = " & swSketchBend.ReverseDirection
    Debug.Print " UseDefaultBendAllowance" = " & swSketchBend.UseDefaultBendAllowance
```

```
Debug.Print "    UseDefaultBendRadius    = " & swSketchBend.UseDefaultBendRadius

' Roll back to get to fixed face
bRet = swSketchBend.AccessSelections(swModel, Nothing): Debug.Assert bRet

Set swFace = swSketchBend.GetFixedFace(nFace_X, nFace_Y, nFace_Z)
Set swEnt = swFace

Debug.Print "    Fixed Face                = (" & nFace_X * 1000# & ", " & nFace_Y * 1000# & ", " & nFace_Z * 1000# & ") mm"

Debug.Print "    Custom Bend Allowance:"
DumpCustomBendAllowanceInfo "    ", swCustBendAllow

bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet

' Make some changes
swSketchBend.UseDefaultBendRadius = False
swSketchBend.BendRadius = swSketchBend.BendRadius * 1.5

' Apply changes
bRet = swFeat.ModifyDefinition(swSketchBend, swModel, Nothing): Debug.Assert bRet
End Sub
'-----
```

Change Color of Component in Specific Display State Example (C#)

This example shows how to change the color of a:

- selected component in a specific display state at the assembly and part levels.
- component in its part document.

```
//-----
// Preconditions: Verify that the specified assembly and part exist.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Examine the graphics area, then press F5.
// 3. Changes the selected component's, USB_cover1,
// assembly-level color from the default red
// to gray in the specified display state. This
// is the overriding color level.
// 4. Examine the component in the graphics area, then press F5.
// 5. Changes the selected component's, USB_cover1, part-level
// color from red to green in the specified display state of
// the component part. This is not the overriding color level.
// 6. Closes, but does not save, the assembly document.
// 7. Opens the component's part document, USB_cover1.sldprt.
// 8. Examine the graphics area, then press F5.
// 9. Changes part's color from red to green in the specified
// display state of the part. This is the overriding color
// level.
// 10. Examine the graphics area, then press F5.
// 11. Closes, but does not save, the part document.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace AppearanceCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc = null;
            ModelDocExtension swModelDocExt = null;
            SelectionMgr swSelMgr = null;
            DisplayStateSetting swDisplayStateSetting = null;
            Component2 swComponent = null;
            Component2[] swComponents = new Component2[1];
            string[] displayStateNames = new string[1];
            object appearances = null;
            object[] appearancesArray = null;
            AppearanceSetting swAppearanceSetting = default(AppearanceSetting);
            AppearanceSetting[] newAppearanceSetting = new AppearanceSetting[1];
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Configuration swConfig = default(Configuration);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = "";
            int red_rgb = 0;
            int green_rgb = 0;
            int blue_rgb = 0;
            int newColor = 0;

            //Open assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS_2024\\samples\\tutorial\\appearances\\usb_flash_drive1.sldasm";
            swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            System.Diagnostics.Debugger.Break();
            //Examine the graphics area
            //Press F5 to continue

            //Select component, USB_cover1
            swModelDocExt = (ModelDocExtension)swModelDoc.Extension;
            swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;
            status = swModelDocExt.SelectByID2("USB_cover1-1@usb_flash_drive1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swComponent = (Component2)swSelMgr.GetSelectedObject6(1, -1);
            swComponents[0] = swComponent;

            swModelDoc.ClearSelection2(true);

            //Get display state
            swDisplayStateSetting = (DisplayStateSetting)swModelDocExt.GetDisplayStateSetting((int)swDisplayStateOpts_e.swAllDisplayState);
            swDisplayStateSetting.Entities = swComponents;
            swDisplayStateSetting.Option = (int)swDisplayStateOpts_e.swSpecifyDisplayState;
            displayStateNames[0] = "Display State-1";
            swDisplayStateSetting.Names = displayStateNames;
            //Assembly level is default
            swDisplayStateSetting.PartLevel = false;

            //Change the selected component's, USB_cover1,
            //assembly-level color from the default red
            //to gray in the specified display state; this
            //is the overriding color level
            appearances = (object)swModelDocExt.get_DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting);
            appearancesArray = (object[])appearances;
            swAppearanceSetting = (AppearanceSetting)appearancesArray[0];
            red_rgb = 50;
            green_rgb = 50;
            blue_rgb = 50;
            newColor = Math.Max(Math.Min(red_rgb, 255), 0) + Math.Max(Math.Min(green_rgb, 255), 0) * 16 * 16 + Math.Max(Math.Min(blue_rgb, 255), 0) * 16 * 16 * 16;
            swAppearanceSetting.Color = newColor;

            swAppearanceSetting.Diffuse = 0.8d;
        }
    }
}
```

```

        swAppearanceSetting.Specular = 0.5d;
        swAppearanceSetting.SpecularColor = newColor;
        swAppearanceSetting.Luminous = 0.1d;
        swAppearanceSetting.Transparent = 0d;

newAppearanceSetting[0] = (AppearanceSetting)swAppearanceSetting;
swModelDocExt.set_DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting, newAppearanceSetting);

System.Diagnostics.Debugger.Break();
//Examine the component in the graphics area
//Press F5 to continue

swModelDoc.ClearSelection2(true);

status = swModelDocExt.SelectByID2("USB_cover1-1@usb_flash_drive1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
swComponent = (Component2)swSelMgr.GetSelectedObject6(1, -1);
swComponents[0] = (Component2)swComponent;

swModelDoc.ClearSelection2(true);

//Get display state
swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting((int)swDisplayStateOpts_e.swAllDisplayState);
swDisplayStateSetting.Entities = swComponents;
swDisplayStateSetting.Option = (int)swDisplayStateOpts_e.swSpecifyDisplayState;
displayStateNames[0] = "<Default> Display State 1";
swDisplayStateSetting.Names = displayStateNames;
//Set to part level
swDisplayStateSetting.PartLevel = true;

//Change the selected component's, USB_cover1, part-level
//color from red to green in the specified display state of
//the component part; this is not the overriding color level
appearances = (object)swModelDocExt.get_DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting);
appearancesArray = (object[])appearances;
swAppearanceSetting = (AppearanceSetting)appearancesArray[0];
red_rgb = 0;
green_rgb = 255;
blue_rgb = 0;
newColor = Math.Max(Math.Min(red_rgb, 255), 0) + Math.Max(Math.Min(green_rgb, 255), 0) * 16 * 16 + Math.Max(Math.Min(blue_rgb, 255), 0) * 16 *
16 * 16 * 16;
swAppearanceSetting.Color = newColor;

        swAppearanceSetting.Diffuse = 0.8d;
        swAppearanceSetting.Specular = 0.5d;
        swAppearanceSetting.SpecularColor = newColor;
        swAppearanceSetting.Luminous = 0.1d;
        swAppearanceSetting.Transparent = 0.0d;

newAppearanceSetting[0] = swAppearanceSetting;
swModelDocExt.set_DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting, newAppearanceSetting);

//Close the assembly document without saving
//changes
swApp.CloseDoc("usb_flash_drive1");

//Open the assembly component USB_cover1 as a part document
fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2024\\samples\\tutorial\\appearances\\usb_cover1.sldprt";
swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref er
rors, ref warnings);
swModelDocExt = (ModelDocExtension)swModelDoc.Extension;
swConfigMgr = (ConfigurationManager)swModelDoc.ConfigurationManager;
swConfig = (Configuration)swConfigMgr.ActiveConfiguration;

swComponents[0] = null;
swComponents[0] = swConfig.GetRootComponent3(true);

swModelDoc.ClearSelection2(true);

//Get display state
swDisplayStateSetting = null;
swDisplayStateSetting = (DisplayStateSetting)swModelDocExt.GetDisplayStateSetting((int)swDisplayStateOpts_e.swAllDisplayState);
swDisplayStateSetting.Entities = swComponents;
swDisplayStateSetting.Option = (int)swDisplayStateOpts_e.swSpecifyDisplayState;
displayStateNames[0] = "<Default> Display State 1";
swDisplayStateSetting.Names = displayStateNames;

System.Diagnostics.Debugger.Break();
//Examine the graphics area
//Press F5 to continue

//Change color of selected component in specified display state
//from default red to green; this is the overriding color
appearances = (object)swModelDocExt.get_DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting);
appearancesArray = (object[])appearances;
swAppearanceSetting = (AppearanceSetting)appearancesArray[0];
red_rgb = 0;
green_rgb = 255;
blue_rgb = 0;
newColor = Math.Max(Math.Min(red_rgb, 255), 0) + Math.Max(Math.Min(green_rgb, 255), 0) * 16 * 16 + Math.Max(Math.Min(blue_rgb, 255), 0) * 16 *
16 * 16 * 16;
swAppearanceSetting.Color = newColor;

        swAppearanceSetting.Diffuse = 0.8d;
        swAppearanceSetting.Specular = 0.5d;
        swAppearanceSetting.SpecularColor = newColor;
        swAppearanceSetting.Luminous = 0.1d;
        swAppearanceSetting.Transparent = 0.0d;

newAppearanceSetting[0] = swAppearanceSetting;
swModelDocExt.set_DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting, newAppearanceSetting);

System.Diagnostics.Debugger.Break();
//Examine the graphics area
//Press F5 to continue

//Close the part document without saving
//changes
swApp.CloseDoc("usb_cover1");

```

```
}  
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
  
}
```

Change Color of Component in Specific Display State Example (VBA)

This example shows how to change the color of a:

- selected component in a specific display state at the assembly and part levels.
- component in its part document.

```
'-----
' Postconditions:
' 1. Verify that the specified assembly and part exist.
' 2. Add the Microsoft Excel Object Library as a reference
'    for the Min and Max functions.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Changes the selected component's, USB_cover1,
'    assembly-level color from the default red
'    to gray in the specified display state. This
'    is the overriding color level.
' 3. To verify, examine the component in the graphics area and
'    click Show Display Pane (>) in the FeatureManager design tree
'    and examine the Appearance column to see the level where the
'    appearance color was applied.
' 4. Press F5 to continue.
' 5. Changes the selected component's, USB_cover1, part-level
'    color from red to green in the specified display state of
'    the component part. This is not the overriding color level.
' 6. To verify, examine the Appearance column to see the
'    level where the appearance color was applied.
' 7. Press F5 to continue.
' 8. Closes, but does not save, the assembly document.
' 9. Opens the component's part document, USB_cover1.sldprt.
' 10. Examine the graphics area.
' 11. Press F5 to continue.
' 12. Changes the part's color from red to green in the specified
'    display state of the part. This is the overriding color
'    level.
' 13. To verify, examine the graphics area.
' 14. Press F5 to continue.
' 15. Closes, but does not save, the part document.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModelDoc As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDisplayStateSetting As SldWorks.DisplayStateSetting
    Dim swComponent As SldWorks.Component2
    Dim swComponents(0) As SldWorks.Component2
    Dim swConfig As SldWorks.Configuration
    Dim displayStateNames(0) As String
    Dim appearances As Variant
    Dim swAppearanceSetting As SldWorks.AppearanceSetting
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim newAppearances(0) As Object
    Dim status As Boolean
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String
    Dim red_rgb As Long
    Dim green_rgb As Long
    Dim blue_rgb As Long
    Dim newColor As Long

    Set swApp = Application.SldWorks

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\appearances\usb_flash_drive1.sldasm"
    Set swModelDoc = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)

    'Select component, USB_cover1
    Set swModelDocExt = swModelDoc.Extension
    Set swSelMgr = swModelDoc.SelectionManager
    status = swModelDocExt.SelectByID2("USB_cover1-l@usb_flash_drive1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComponent = swSelMgr.GetSelectedObject6(1, -1)
    Set swComponents(0) = swComponent

    swModelDoc.ClearSelection2 True

    'Get display state
    Set swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting(swAllDisplayState)
    swDisplayStateSetting.Entities = swComponents
    swDisplayStateSetting.Option = swSpecifyDisplayState
    displayStateNames(0) = "Display State-1"
    swDisplayStateSetting.Names = displayStateNames

    'Assembly level is the default
    swDisplayStateSetting.PartLevel = False
```

```

'Change the selected component's, USB_cover1,
'assembly-level color from the default red
'to gray in the specified display state; this
'is the overriding color level
appearances = swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting)
Set swAppearanceSetting = appearances(0)
red_rgb = 50
green_rgb = 50
blue_rgb = 50
newColor = Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(red_rgb, 255), 0) +
Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(green_rgb, 255), 0) * 16 * 16 + Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(blue_rgb,
255), 0) * 16 * 16 * 16 * 16
swAppearanceSetting.Color = newColor

swAppearanceSetting.Diffuse = 0.8
swAppearanceSetting.Specular = 0.5
swAppearanceSetting.SpecularColor = newColor
swAppearanceSetting.Luminous = 0.1
swAppearanceSetting.Transparent = 0.0

Set newAppearances(0) = swAppearanceSetting
swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting) = newAppearances

Stop
'Examine the component in the graphics area
'Click Show Display Pane (>) in the FeatureManager
'design tree and examine the Appearance column to see
'the level where the appearance color was applied
'Press F5 to continue

swModelDoc.ClearSelection2 True

status = swModelDocExt.SelectByID2("USB_cover1-1@usb_flash_drive1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
Set swComponent = swSelMgr.GetSelectedObject6(1, -1)
Set swComponents(0) = swComponent

swModelDoc.ClearSelection2 True

'Get display state
Set swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting(swAllDisplayState)
swDisplayStateSetting.Entities = swComponents
swDisplayStateSetting.Option = swSpecifyDisplayState
displayStateNames(0) = "<Default> Display State 1"
swDisplayStateSetting.Names = displayStateNames
'Set to part level
swDisplayStateSetting.PartLevel = True

'Change the selected component's, USB_cover1, part-level
'color from red to green in the specified display state of
'the component part; this is not the overriding color level
appearances = swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting)
Set swAppearanceSetting = appearances(0)
red_rgb = 0
green_rgb = 255
blue_rgb = 0
newColor = Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(red_rgb, 255), 0) +
Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(green_rgb, 255), 0) * 16 * 16 + Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(blue_rgb,
255), 0) * 16 * 16 * 16 * 16
swAppearanceSetting.Color = newColor

swAppearanceSetting.Diffuse = 0.8
swAppearanceSetting.Specular = 0.5
swAppearanceSetting.SpecularColor = newColor
swAppearanceSetting.Luminous = 0.1
swAppearanceSetting.Transparent = 0.0

Set newAppearances(0) = swAppearanceSetting
swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting) = newAppearances

Stop
'Examine the Appearance column to see the level
'where the appearance color was applied
'Press F5 to continue

'Close the assembly document without saving
'changes
swApp.CloseDoc "usb_flash_drive1"

'Open the assembly component USB_cover1 as a part document
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\appearances\usb_cover1.sldprt"
Set swModelDoc = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

Set swModelDocExt = swModelDoc.Extension
Set swSelMgr = swModelDoc.SelectionManager
Set swConfigMgr = swModelDoc.ConfigurationManager
Set swConfig = swConfigMgr.ActiveConfiguration

Set swComponents(0) = Nothing
Set swComponents(0) = swConfig.GetRootComponent3(True)

```

```

swModelDoc.ClearSelection2 True

'Get display state
Set swDisplayStateSetting = Nothing
Set swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting(swAllDisplayState)
swDisplayStateSetting.Entities = swComponents
swDisplayStateSetting.Option = swSpecifyDisplayState
displayStateNames(0) = "<Default>_Display State 1"
swDisplayStateSetting.Names = displayStateNames

Stop
'Examine the graphics area
'Press F5 to continue

'Change color of selected component in specified display state
'from default red to green; this is the overriding color
appearances = swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting)
Set swAppearanceSetting = Nothing
Set swAppearanceSetting = appearances(0)
red_rgb = 0
green_rgb = 255
blue_rgb = 0
newColor = Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(red_rgb, 255), 0) +
Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(green_rgb, 255), 0) * 16 * 16 + Excel.WorksheetFunction.Max(Excel.WorksheetFunction.Min(blue_rgb,
255), 0) * 16 * 16 * 16 * 16
swAppearanceSetting.Color = newColor

swAppearanceSetting.Diffuse = 0.8
swAppearanceSetting.Specular = 0.5
swAppearanceSetting.SpecularColor = newColor
swAppearanceSetting.Luminous = 0.1
swAppearanceSetting.Transparent = 0.0

Set newAppearances(0) = swAppearanceSetting
swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting) = newAppearances

Stop
'Examine the graphics area
'Press F5 to continue

'Close the part document without saving
'changes
swApp.CloseDoc "usb_cover1"

End Sub

```

Change Color of Component in Specific Display State Example (VB.NET)

This example shows how to change the color of a:

- selected component in a specific display state at the assembly and part levels.
- component in its part document.

```

'-----
' Postconditions: Verify that the specified assembly and part exist.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Examine the graphics areas, then press F5.
' 3. Changes the selected component's, USB_cover1,
'    assembly-level color from the default red
'    to gray in the specified display state. This
'    is the overriding color level.
' 4. Examine the component in the graphics area, then press F5.
' 5. Changes the selected component's, USB_cover1, part-level
'    color from red to green in the specified display state of
'    the component part. This is not the overriding color level.
' 6. Closes, but does not save, the assembly document.
' 7. Opens the component's part document, USB_cover1.sldprt.
' 8. Examine the graphics area, then press F5.
' 9. Changes the part's color from red to green in the specified
'    display state of the part. This is the overriding color
'    level.
' 10. Examine the graphics area, then press F5.
' 11. Closes, but does not save, the part document.
'-----

```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System.Runtime.InteropServices

Imports System

Partial Class SolidWorksMacro

Public Sub Main()

```

    Dim swModelDoc As ModelDoc2 = Nothing
    Dim swModelDocExt As ModelDocExtension = Nothing
    Dim swSelMgr As SelectionMgr = Nothing
    Dim swDisplayStateSetting As DisplayStateSetting = Nothing
    Dim swComponent As Component2 = Nothing
    Dim swComponents(0) As Component2
    Dim displayStateNames(0) As String
    Dim appearances As Object
    Dim swAppearanceSetting As AppearanceSetting
    Dim newAppearanceSetting(0) As AppearanceSetting
    Dim swConfigMgr As ConfigurationManager
    Dim swConfig As Configuration
    Dim status As Boolean = False
    Dim errors As Integer = 0
    Dim warnings As Integer = 0
    Dim fileName As String = ""
    Dim red_rgb As Integer = 0
    Dim green_rgb As Integer = 0
    Dim blue_rgb As Integer = 0
    Dim newColor As Integer = 0

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\appearances\usb_flash_drive1.sldasm"
    swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    Stop
    'Examine the graphics area
    'Press F5 to continue

    'Select component, USB_cover1
    swModelDocExt = swModelDoc.Extension
    swSelMgr = swModelDoc.SelectionManager
    status = swModelDocExt.SelectByID2("USB_cover1-1@usb_flash_drive1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swComponent = swSelMgr.GetSelectedObject6(1, -1)
    swComponents(0) = swComponent

    swModelDoc.ClearSelection2(True)

    'Get display state
    swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting(swDisplayStateOpts_e.swAllDisplayState)
    swDisplayStateSetting.Entities = swComponents
    swDisplayStateSetting.Option = swDisplayStateOpts_e.swSpecifyDisplayState
    displayStateNames(0) = "Display State-1"
    swDisplayStateSetting.Names = displayStateNames
    'Assembly level is default
    swDisplayStateSetting.PartLevel = False

    'Change the selected component's, USB_cover1,
    'assembly-level color from the default red
    'to gray in the specified display state; this
    'is the overriding color level
    appearances = swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting)
    swAppearanceSetting = appearances(0)
    red_rgb = 50
    green_rgb = 50

```

```

blue_rgb = 50
newColor = Math.Max(Math.Min(red_rgb, 255), 0) + Math.Max(Math.Min(green_rgb, 255), 0) * 16 * 16 + Math.Max(Math.Min(blue_rgb, 255), 0) * 16 * 16
* 16 * 16
swAppearanceSetting.Color = newColor

swAppearanceSetting.Diffuse = 0.8
swAppearanceSetting.Specular = 0.5
swAppearanceSetting.SpecularColor = newColor
swAppearanceSetting.Luminous = 0.1
swAppearanceSetting.Transparent = 0.0

newAppearanceSetting(0) = swAppearanceSetting
swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting) = newAppearanceSetting

Stop
'Examine the component in the graphics area
'Press F5 to continue

swModelDoc.ClearSelection2(True)

status = swModelDocExt.SelectByID2("USB_cover1-1@usb_flash_drive1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
swComponent = swSelMgr.GetSelectedObject6(1, -1)
swComponents(0) = swComponent

swModelDoc.ClearSelection2(True)

'Get display state
swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting(swDisplayStateOpts_e.swAllDisplayState)
swDisplayStateSetting.Entities = swComponents
swDisplayStateSetting.Option = swDisplayStateOpts_e.swSpecifyDisplayState
displayStateNames(0) = "<Default>_Display State 1"
swDisplayStateSetting.Names = displayStateNames
'Set to part level
swDisplayStateSetting.PartLevel = True

'Change the selected component's, USB_cover1, part-level
'color from red to green in the specified display state of
'the component part; this is not the overriding color level
appearances = swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting)
swAppearanceSetting = appearances(0)
red_rgb = 0
green_rgb = 255
blue_rgb = 0
newColor = Math.Max(Math.Min(red_rgb, 255), 0) + Math.Max(Math.Min(green_rgb, 255), 0) * 16 * 16 + Math.Max(Math.Min(blue_rgb, 255), 0) * 16 * 16
* 16 * 16
swAppearanceSetting.Color = newColor

swAppearanceSetting.Diffuse = 0.8
swAppearanceSetting.Specular = 0.5
swAppearanceSetting.SpecularColor = newColor
swAppearanceSetting.Luminous = 0.1
swAppearanceSetting.Transparent = 0.0

newAppearanceSetting(0) = swAppearanceSetting
swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting) = newAppearanceSetting

'Close the assembly document without saving
'changes
swApp.CloseDoc("usb_flash_drive1")

'Open the assembly component USB_cover1 as a part document
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\appearances\usb_cover1.sldprt"
swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

swModelDocExt = swModelDoc.Extension
swSelMgr = swModelDoc.SelectionManager
swConfigMgr = swModelDoc.ConfigurationManager
swConfig = swConfigMgr.ActiveConfiguration

swComponents(0) = Nothing
swComponents(0) = swConfig.GetRootComponent3(True)

swModelDoc.ClearSelection2(True)

'Get display state
swDisplayStateSetting = Nothing
swDisplayStateSetting = swModelDocExt.GetDisplayStateSetting(swDisplayStateOpts_e.swAllDisplayState)
swDisplayStateSetting.Entities = swComponents
swDisplayStateSetting.Option = swDisplayStateOpts_e.swSpecifyDisplayState
displayStateNames(0) = "<Default>_Display State 1"
swDisplayStateSetting.Names = displayStateNames

Stop
'Examine the graphics area
'Press F5 to continue

'Change color of selected component in specified display state
'from default red to green; this is the overriding color
appearances = swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting)
swAppearanceSetting = Nothing
swAppearanceSetting = appearances(0)
red_rgb = 0
green_rgb = 255
blue_rgb = 0
newColor = Math.Max(Math.Min(red_rgb, 255), 0) + Math.Max(Math.Min(green_rgb, 255), 0) * 16 * 16 + Math.Max(Math.Min(blue_rgb, 255), 0) * 16 * 16
* 16 * 16
swAppearanceSetting.Color = newColor

swAppearanceSetting.Diffuse = 0.8

```

```
swAppearanceSetting.Specular = 0.5
swAppearanceSetting.SpecularColor = newColor
swAppearanceSetting.Luminous = 0.1
swAppearanceSetting.Transparent = 0.0

newAppearanceSetting(0) = swAppearanceSetting
swModelDocExt.DisplayStateSpecMaterialPropertyValues(swDisplayStateSetting) = newAppearanceSetting
```

```
Stop
```

```
'Examine the graphics area
'Press F5 to continue
```

```
'Close the part document without saving
'changes
swApp.CloseDoc("usb_cover1")
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Change Color of Face Example (VBA)

This example shows how to change the color of the selected face to blue.

```
'-----
' Preconditions:
' 1. Open a part and select a face.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Changes the color of the selected face to blue.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim vFaceProp As Variant
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)

    vFaceProp = swFace.MaterialPropertyValues

    If IsEmpty(vFaceProp) Then
        ' Is empty if face-level colors were not specified,
        ' so get them from underlying model
        vFaceProp = swModel.MaterialPropertyValues
    End If

    'Current color
    Debug.Print "RGB" = "[" & vFaceProp(0) * 255# & ", " & vFaceProp(1) * 255# & ", " & vFaceProp(2) * 255# & "]"
    Debug.Print "Ambient" = " & vFaceProp(3)
    Debug.Print "Diffuse" = " & vFaceProp(4)
    Debug.Print "Specular" = " & vFaceProp(5)
    Debug.Print "Shininess" = " & vFaceProp(6)
    Debug.Print "Transparency" = " & vFaceProp(7)
    Debug.Print "Emission" = " & vFaceProp(8)

    ' New color
    bRet = swModel.SelectedFaceProperties(RGB(0, 0, 255), vFaceProp(3), vFaceProp(4), vFaceProp(5), vFaceProp(6), vFaceProp(7), vFaceProp(8), False, "")

    ' Deselect face to see new color
    swModel.ClearSelection2 True

End Sub
```

Change Component to Envelope Example (C#)

This example shows how to change a component to an envelope.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects the shaft washer-4 component.
// 3. Gets whether the component is an envelope.
// 4. Changes the component to an envelope.
// 5. Gets whether the component is an envelope.
// 6. Examine the Immediate window and FeatureManager design tree.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            Component2 swComponent = default(Component2);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\98food_processor.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);

            //Select shaft washer-4 component
            swModelDocExtension = (ModelDocExtension)swModel.Extension;
            status = swModelDocExtension.SelectByID2("shaft washer-4@98food_processor", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swComponent = (Component2)swSelectionMgr.GetSelectedObjectsComponent4(1, -1);

            //Get whether selected component is an envelope
            Debug.Print("Before calling IAssemblyDoc::CompConfigProperties5:");
            Debug.Print(" Is component an envelope? " + swComponent.IsEnvelope());

            //Change the selected component to an envelope
            swAssembly = (AssemblyDoc)swModel;
            status = swAssembly.CompConfigProperties6((int)swComponentSuppressionState_e.swComponentFullyResolved, (int)swComponentSolvingOption_e.swCompo
            nentRigidSolving, true, true, "Default", false, true, (int)swASMSLDPRTCompPref_e.swAlwaysInclude);

            //Get whether the selected component is an envelope
            Debug.Print("After calling IAssemblyDoc::CompConfigProperties5:");
            Debug.Print(" Is component an envelope? " + swComponent.IsEnvelope());

            swModel.EditRebuild3();
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Change Component to Envelope Example (VBA)

This example shows how to change a component to an envelope.

```
'-----  
' Preconditions:  
' 1. Verify that the specified assembly to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the assembly.  
' 2. Selects the shaft washer-4 component.  
' 3. Gets whether the component is an envelope.  
' 4. Changes the component to an envelope.  
' 5. Gets whether the component is an envelope.  
' 6. Examine the Immediate window and FeatureManager design tree.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExtension As SldWorks.ModelDocExtension  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim swComponent As SldWorks.Component2  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food processor.sldasm"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
  
    'Select shaft washer-4 component  
    Set swModelDocExtension = swModel.Extension  
    status = swModelDocExtension.SelectByID2("shaft washer-4@98food processor", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swComponent = swSelectionMgr.GetSelectedObjectsComponent4(1, -1)  
  
    'Get whether selected component is an envelope  
    Debug.Print "Before calling IAssemblyDoc::CompConfigProperties5:"  
    Debug.Print "    Is component an envelope? " & swComponent.IsEnvelope  
  
    'Change the selected component to an envelope  
    Set swAssembly = swModel  
    status = swAssembly.CompConfigProperties6(swComponent.SuppressionState_e.swComponentFullyResolved, swComponent.SolvingOption_e.swComponentRigidSolving,  
True, True, "Default", False, True, swASMSLDPRTCompPref_e.swAlwaysInclude)  
  
    'Get whether the selected component is an envelope  
    Debug.Print "After calling IAssemblyDoc::CompConfigProperties5:"  
    Debug.Print "    Is component an envelope? " & swComponent.IsEnvelope  
  
    swModel.EditRebuild3  
  
End Sub
```

Change Component to Envelope Example (VB.NET)

This example shows how to change a component to an envelope.

```

-----
Preconditions:
1. Verify that the specified assembly to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the assembly.
2. Selects the shaft washer-4 component.
3. Gets whether the component is an envelope.
4. Changes the component to an envelope.
5. Gets whether the component is an envelope.
6. Examine the Immediate window and FeatureManager design tree.
NOTE: Because the assembly is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExtension As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swAssembly As AssemblyDoc
        Dim swComponent As Component2
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food_processor.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Select shaft washer-4 component
        swModelDocExtension = swModel.Extension
        status = swModelDocExtension.SelectByID2("shaft washer-4@98food_processor", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swComponent = swSelectionMgr.GetSelectedObjectsComponent4(1, -1)

        'Get whether selected component is an envelope
        Debug.Print("Before calling IAssemblyDoc::CompConfigProperties5:")
        Debug.Print("  Is component an envelope? " & swComponent.IsEnvelope)

        'Change the selected component to an envelope
        swAssembly = swModel
        status = swAssembly.CompConfigProperties6(swComponent.SuppressionState_e.swComponentFullyResolved, swComponent.SolvingOption_e.swComponentRigidSolving, True, True, "Default", False, True, swASMLDPRTPCompPref_e.swAlwaysInclude)

        'Get whether the selected component is an envelope
        Debug.Print("After calling IAssemblyDoc::CompConfigProperties5:")
        Debug.Print("  Is component an envelope? " & swComponent.IsEnvelope)

        swModel.EditRebuild3()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Change Configuration Properties Example (VBA)

This example shows how to change the properties of one or more configurations.

```

'-----
' Preconditions:
' 1. Open a part that contains one or more configurations.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Changes the source for the part number and displays XXXX
'    as the part number in each configuration's Bill of Materials.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.PartDoc
Dim Doc As SldWorks.ModelDoc2
Dim SelMgr As SldWorks.SelectionMgr

Function BOMPartNumber(config As SldWorks.Configuration, document As SldWorks.ModelDoc2) As String
    Select Case config.BOMPartNoSource
    Case SwConst.swBOMPartNumberSource_e.swBOMPartNumber_ConfigurationName
        BOMPartNumber = config.Name
    Case SwConst.swBOMPartNumberSource_e.swBOMPartNumber_DocumentName
        BOMPartNumber = document.GetTitle
    Case SwConst.swBOMPartNumberSource_e.swBOMPartNumber_UserSpecified
        BOMPartNumber = config.AlternateName
    Case SwConst.swBOMPartNumberSource_e.swBOMPartNumber_ParentName
        Dim parentConfig As SldWorks.Configuration
        Set parentConfig = config.GetParent
        If parentConfig.BOMPartNoSource = SwConst.swBOMPartNumberSource_e.swBOMPartNumber_ParentName Then
            BOMPartNumber = BOMPartNumber(parentConfig, document)
        Else
            BOMPartNumber = parentConfig.Name
        End If
    End Select
End Function

Function InspectConfigurations(Doc As SldWorks.ModelDoc2)
    Dim params As Variant
    params = Doc.GetConfigurationNames
    Dim vName As Variant
    Dim Name As String
    Dim thisConfig As Configuration
    For Each vName In params
        Name = vName
        Set thisConfig = Doc.GetConfigurationByName(Name)
        Debug.Print "Name", thisConfig.Name
        ' Work out what the BOM part number is based on any derived configurations
        Debug.Print "BOMPartNumber", BOMPartNumber(thisConfig, Doc)
        Debug.Print "AlternateName", thisConfig.AlternateName
        Debug.Print "Comment", thisConfig.Comment
        Debug.Print "Description", thisConfig.Description
        Debug.Print "HideNewComponentModels", thisConfig.HideNewComponentModels
        Debug.Print "Lock", thisConfig.Lock
        Debug.Print "ShowChildComponentsInBOM", thisConfig.ShowChildComponentsInBOM
        Debug.Print "UseAlternateNameInBOM", thisConfig.UseAlternateNameInBOM
        Debug.Print "SuppressNewComponentModels", thisConfig.SuppressNewComponentModels
        Debug.Print "SuppressNewFeatures", thisConfig.SuppressNewFeatures
        Debug.Print "-----"
    Next vName
End Function

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set Doc = Part
    Set SelMgr = Doc.SelectionManager

    Call InspectConfigurations(Doc)

    Dim params As Variant
    params = Doc.GetConfigurationNames
    Dim vName As Variant
    Dim Name As String
    Dim thisConfig As Configuration
    Debug.Print "Modifying the configurations..."
    For Each vName In params
        Name = vName
        Set thisConfig = Doc.GetConfigurationByName(Name)
        Debug.Print "Name", thisConfig.Name
        thisConfig.BOMPartNoSource = swBOMPartNumber_UserSpecified
        thisConfig.AlternateName = "XXXX"
        thisConfig.UseAlternateNameInBOM = True
        thisConfig.AlternateName = "XXXX"
    Next vName

    Debug.Print "-----"

    Call InspectConfigurations(Doc)

End Sub

```

Change Dimension Example (C#)

This example shows how to change a dimension value in a model.

NOTE: Most of the SOLIDWORKS API functions operate in meters. Thus, if you pass in XValue_Passed = 2.0 and your model units are millimeters, then it appears as a 2000.0 in the model. If you need to determine the units used in the model, you can use the IModelDoc2::LengthUnit property and perform the appropriate conversion.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Changes the specified dimension parameter of the selected feature.
// 3. Examine the Immediate window.
//
// NOTE: Because the assembly document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Feature swFeature;
        SelectionMgr swSelectionManager;
        Dimension swDim;
        string fileName;
        bool boolstatus;
        int errors;
        int warnings;

        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assem2.sldasm";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            boolstatus = swModel.Extension.SelectByID2("LocalCirPattern1", "COMPATTERN", 0, 0, 0, false, 0, null, 0);
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionManager.GetSelectedObject6(1, -1);

            swDim = (Dimension)swFeature.Parameter("D3");
            Debug.Print("D3@LocalCirPattern1 is " + swDim.SystemValue.ToString() + " before changing it.");

            // Change D3 of LocalCirPattern1 from 360 degrees to 270 degrees (4.72 radians)
            errors = swDim.SetSystemValue3(4.72, (int)swSetValueInConfiguration_e.swSetValue_InThisConfiguration, null);

            swModel.EditRebuild3();

            Debug.Print("D3@LocalCirPattern1 is " + swDim.SystemValue.ToString() + " after changing it.");
        }

        public SldWorks swApp;
    }
}
```

Change Dimension Example (VBA)

This example shows how to change a dimension value in a model.

NOTE: Most of the SOLIDWORKS API functions operate in meters. Thus, if you pass in XValue_Passed = 2.0 and your model units are millimeters, then it appears as a 2000.0 in the model. If you need to determine the units used in the model, you can use the IModelDoc2::LengthUnit property and perform the appropriate conversion.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Changes the specified dimension parameter of the selected feature.
' 3. Examine the Immediate window.
'
' NOTE: Because the assembly document is used elsewhere,
' do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeature As SldWorks.Feature
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swDim As SldWorks.Dimension
Dim fileName As String
Dim boolstatus As Boolean
Dim errors As Long
Dim warnings As Long
Dim val As Variant

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)

    boolstatus = swModel.Extension.SelectByID2("LocalCirPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionManager = swModel.SelectionManager
    Set swFeature = swSelectionManager.GetSelectedObject6(1, -1)

    Set swDim = swFeature.Parameter("D3")

    ' Get D3 of LocalCirPattern1
    Debug.Print "D3@LocalCirPattern1 is " & swFeature.Parameter("D3").SystemValue & " before changing it."

    ' Change D3 of LocalCirPattern1 from 360 degrees to 270 degrees (4.72 radians)
    errors = swDim.SetSystemValue3(4.72, swSetValue_InThisConfiguration, Empty)

    swModel.EditRebuild3

    Debug.Print "D3@LocalCirPattern1 is " & swFeature.Parameter("D3").SystemValue & " after changing it."

End Sub
```

Change Dimension Example (VB.NET)

This example shows how to change a dimension value in a model.

NOTE: Most of the SOLIDWORKS API functions operate in meters. Thus, if you pass in XValue_Passed = 2.0 and your model units are millimeters, then it appears as a 2000.0 in the model. If you need to determine the units used in the model, you can use the IModelDoc2::LengthUnit property and perform the appropriate conversion.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Changes the specified dimension parameter of the selected feature.
' 3. Examine the Immediate window.
'
' NOTE: Because the assembly document is used elsewhere,
' do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swFeature As Feature
    Dim swSelectionManager As SelectionMgr
    Dim swDim As Dimension
    Dim fileName As String
    Dim boolstatus As Boolean
    Dim errors As Integer
    Dim warnings As Integer

    Sub main()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
        errors, warnings)

        boolstatus = swModel.Extension.SelectByID2("LocalCirPattern1", "COMPPATTERN",
        0, 0, 0, False,
        0, Nothing,
        0)

        swSelectionManager = swModel.SelectionManager
        swFeature = swSelectionManager.GetSelectedObject6(1, -1)

        swDim = swFeature.Parameter("D3")

        ' Get D3 of LocalCirPattern1
        Debug.Print("D3@LocalCirPattern1 is " & swDim.SystemValue & " before changing it.")

        ' Change D3 of LocalCirPattern1
        from 360 degrees to 270 degrees (4.72 radians)

        errors = swDim.SetSystemValue3(4.72, swSetValueInConfiguration_e.swSetValue_InThisConfiguration, Nothing)

        swModel.EditRebuild3()

        swDim = swFeature.Parameter("D3")
        Debug.Print("D3@LocalCirPattern1 is " & swDim.SystemValue & " after changing it.")

    End Sub

```

Public swApp As StdWorks

End Class

Change Dimension Tolerance in a Configuration Example (C#)

This example shows how to change the dimension tolerance in one configuration in a multi-configuration part.

```
//-----
// Preconditions:
// 1. Ensure that the specified part document exists.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Opens specified document.
// 2. Selects a sketch and a dimension
//    in that sketch.
// 3. Changes the tolerance values of the selected
//    dimension in the sketch and prints the values
//    to the Immediate window.
// 4. Changes configuration.
// 5. Selects the same sketch and dimension
//    in the sketch in this configuration.
// 6. Prints the tolerance values of the dimension
//    to the Immediate window.
// 7. Examines the Immediate window to verify that
//    the tolerance values of the sketch in the
//    different configurations are different.
//
// NOTE: Because this part document is used elsewhere,
// do not save any changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SetValues2CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            ConfigurationManager swConfigurationMgr = default(ConfigurationManager);
            Configuration swConfiguration = default(Configuration);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            Dimension swDimension = default(Dimension);
            DimensionTolerance swDimensionTolerance = default(DimensionTolerance);
            bool status = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            // Open part document with multiple configurations
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\PDMWorks\\speaker_frame.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Get name of active configuration
            swConfigurationMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfiguration = (Configuration)swConfigurationMgr.ActiveConfiguration;
            Debug.Print("Configuration name: " + swConfiguration.Name);

            // Select sketch
            // Put the sketch in edit mode
            // Select a dimension in the sketch
            status = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.EditSketch();
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("D4@Sketch8@speaker_frame.SLDPRT", "DIMENSION", -0.00430195952926557, 0.0321813003735837, -0.0155776956607312, false, 0, null, 0);

            // Get the selection
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, 0);

            // If selection is not a display dimension, then exit
            if (swSelMgr.GetSelectedObjectType3(1, -1) != (int)swSelectType_e.swSelDIMENSIONS)
                return;

            // Selection is a dimension, so get the dimension tolerance
            swDimension = (Dimension)swDisplayDimension.GetDimension2(0);
            swDimensionTolerance = (DimensionTolerance)swDimension.Tolerance;

            // Set type of tolerance type
            swDimensionTolerance.Type = (int)swTolType_e.swTolBASIC;

            // Set new dimension tolerance values
            status = swDimensionTolerance.SetValues2(0.01, 0.015, (int)swSetValueInConfiguration_e.swSetValue_InThisConfiguration, "");
            Debug.Print(" Minimum dimension tolerance: " + swDimensionTolerance.GetMinValue());
            Debug.Print(" Maximum dimension tolerance: " + swDimensionTolerance.GetMaxValue());

            // Exit sketch edit mode
            swModel.InsertSketch2(true);

            // Switch configuration to verify
            // that dimension tolerance changed
            // in other configuration only
            status = swModel.ShowConfiguration2("Square Cutout Glueable");
            status = swModelDocExt.SelectByID2("Square Cutout Glueable", "CONFIGURATIONS", 0, 0, 0, false, 0, null, 0);

            // Get name of configuration
            swConfiguration = (Configuration)swConfigurationMgr.ActiveConfiguration;
            Debug.Print("Configuration name: " + swConfiguration.Name);

            // Select sketch
            // Select same dimension in sketch as selected
            // in previously active configuration
        }
    }
}
```

```

// Put the sketch in edit mode
status = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, false, 0, null, 0);
swModel.EditSketch();
swModel.ClearSelection2(true);
status = swModelDocExt.SelectByID2("D4@Sketch8@speaker_frame.SLDPRT", "DIMENSION", -0.00471220094479408, 0.032305394835097, -0.0153009205936774, false, 0, null, 0);

// Get the selection
swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, 0);

// If selection is not a display dimension, then exit
if (swSelMgr.GetSelectedObjectType3(1, -1) != (int)swSelectType_e.swSelDIMENSIONS)
    return;

// Selection is a dimension, so get and print the dimension tolerance
swDimension = (Dimension)swDisplayDimension.GetDimension2(0);
swDimensionTolerance = (DimensionTolerance)swDimension.Tolerance;
Debug.Print(" Minimum dimension tolerance: " + swDimensionTolerance.GetMinValue());
Debug.Print(" Maximum dimension tolerance: " + swDimensionTolerance.GetMaxValue());

// Exit sketch edit mode
swModel.InsertSketch2(true);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;

}
}

```

Change Dimension Tolerance in a Configuration Example (VBA)

This example shows how to change the dimension tolerance in one configuration in a multi-configuration part.

```
'-----
' Preconditions:
' 1. Ensure that the specified part document exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens specified document.
' 2. Selects a sketch and a dimension
'   in that sketch.
' 3. Changes the tolerance values of the selected
'   dimension in the sketch and prints the values
'   to the Immediate window.
' 4. Changes configuration.
' 5. Selects the same sketch and dimension
'   in the sketch in this configuration.
' 6. Prints the tolerance values of the dimension
'   to the Immediate window.
' 7. Examine the Immediate window to verify that
'   the tolerance values of the sketch in the
'   different configurations are different.
'
' NOTE: Because this part document is used elsewhere,
' do not save any changes when closing it.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swConfigurationMgr As SldWorks.ConfigurationManager
Dim swConfiguration As SldWorks.Configuration
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim swDimension As SldWorks.Dimension
Dim swDimensionTolerance As SldWorks.DimensionTolerance
Dim status As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open part document with multiple configurations
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\2018\samples\tutorial\PDMWorks\speaker_frame.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    ' Get name of active configuration
    Set swConfigurationMgr = swModel.ConfigurationManager
    Set swConfiguration = swConfigurationMgr.ActiveConfiguration
    Debug.Print ("Configuration name: " & swConfiguration.Name)

    ' Select sketch
    ' Put the sketch in edit mode
    ' Select a dimension in the sketch
    status = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditSketch
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("D4@Sketch8@speaker_frame.SLDPR", "DIMENSION", -4.30195952926557E-03, 3.21813003735837E-02, -1.55776956607312E-02,
    False, 0, Nothing, 0)

    ' Get the selection
    Set swSelMgr = swModel.SelectionManager
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, 0)

    ' If selection is not a display dimension, then exit
    If swSelMgr.GetSelectedObjectType3(1, -1) <> swSelDIMENSIONS Then Exit Sub

    ' Selection is a dimension, so get the dimension tolerance
    Set swDimension = swDisplayDimension.GetDimension2(0)
    Set swDimensionTolerance = swDimension.Tolerance

    ' Set type of tolerance type
    swDimensionTolerance.Type = swTolBASIC

    ' Set new dimension tolerance values
    status = swDimensionTolerance.SetValues2(0.01, 0.015, swSetValueInConfiguration_e.swSetValue_InThisConfiguration, "")
    Debug.Print (" Minimum dimension tolerance: " & swDimensionTolerance.GetMinValue)
    Debug.Print (" Maximum dimension tolerance: " & swDimensionTolerance.GetMaxValue)

    ' Exit sketch edit mode
    swModel.InsertSketch2 True

    ' Switch configuration to verify
    ' that dimension tolerance changed
    ' in other configuration only
    status = swModel.ShowConfiguration2("Square Cutout Glueable")
    status = swModelDocExt.SelectByID2("Square Cutout Glueable", "CONFIGURATIONS", 0, 0, 0, False, 0, Nothing, 0)
```

```

' Get name of configuration
Set swConfiguration = swConfigurationMgr.ActiveConfiguration
Debug.Print ("Configuration name: " & swConfiguration.Name)

' Select sketch
' Select same dimension in sketch as selected
' in previously active configuration
' Put the sketch in edit mode
status = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
swModel.EditSketch
swModel.ClearSelection2 True
status = swModelDocExt.SelectByID2("D4@Sketch8@speaker_frame.SLDPRT", "DIMENSION", -4.71220094479408E-03, 0.032305394835097, -1.53009205936774E-02,
False, 0, Nothing, 0)

' Get the selection
Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, 0)

' If selection is not a display dimension, then exit
If swSelMgr.GetSelectedObjectType3(1, -1) <> swSelDIMENSIONS Then Exit Sub

' Selection is a dimension, so get and print the dimension tolerance
Set swDimension = swDisplayDimension.GetDimension2(0)
Set swDimensionTolerance = swDimension.Tolerance
Debug.Print (" Minimum dimension tolerance: " & swDimensionTolerance.GetMinValue)
Debug.Print (" Maximum dimension tolerance: " & swDimensionTolerance.GetMaxValue)

' Exit sketch edit mode
swModel.InsertSketch2 True

End Sub

```

Change Dimension Tolerance in a Configuration Example (VB.NET)

This example shows how to change the dimension tolerance in one configuration in a multi-configuration part.

```

'-----
' Preconditions:
' 1. Ensure that the specified part document exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'-----
' Postconditions:
' 1. Opens specified document.
' 2. Selects a sketch and a dimension
'   in that sketch.
' 3. Changes the tolerance values of the selected
'   dimension in the sketch and prints the values
'   to the Immediate window.
' 4. Changes configuration.
' 5. Selects the same sketch and dimension
'   in the sketch in this configuration.
' 6. Prints the tolerance values of the dimension
'   to the Immediate window.
' 7. Examine the Immediate window to verify that
'   the tolerance values of the sketch in the
'   different configurations are different.
'-----
' NOTE: Because this part document is used elsewhere,
' do not save any changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swConfigurationMgr As ConfigurationManager
        Dim swConfiguration As Configuration
        Dim swSelMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim swDimension As Dimension
        Dim swDimensionTolerance As DimensionTolerance
        Dim status As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        ' Open part document with multiple configurations
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\2018\samples\tutorial\PDMWorks\speaker_frame.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        ' Get name of active configuration
        swConfigurationMgr = swModel.ConfigurationManager
        swConfiguration = swConfigurationMgr.ActiveConfiguration
        Debug.Print("Configuration name: " & swConfiguration.Name)

        ' Select sketch
        ' Put the sketch in edit mode
        ' Select a dimension in the sketch
        status = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSketch()
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("D4@Sketch8@speaker_frame.SLDPRT", "DIMENSION", -0.00430195952926557, 0.0321813003735837, -0.0155776956607312, False, 0, Nothing, 0)

        ' Get the selection
        swSelMgr = swModel.SelectionManager
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, 0)

        ' If selection is not a display dimension, then exit
        If swSelMgr.GetSelectedObjectType3(1, -1) <> swSelectType_e.swSelDIMENSIONS Then Exit Sub

        ' Selection is a dimension, so get the dimension tolerance
        swDimension = swDisplayDimension.GetDimension2(0)
        swDimensionTolerance = swDimension.Tolerance

        ' Set type of tolerance type
        swDimensionTolerance.Type = swTolType_e.swTolBASIC

        ' Set new dimension tolerance values
        status = swDimensionTolerance.SetValues2(0.01, 0.015, swSetValueInConfiguration_e.swSetValue_InThisConfiguration, "")
        Debug.Print(" Minimum dimension tolerance: " & swDimensionTolerance.GetMinValue)
        Debug.Print(" Maximum dimension tolerance: " & swDimensionTolerance.GetMaxValue)

        ' Exit sketch edit mode
        swModel.InsertSketch2(True)

        ' Switch configuration to verify
        ' that dimension tolerance changed
        ' in other configuration only
        status = swModel.ShowConfiguration2("Square Cutout Glueable")
        status = swModelDocExt.SelectByID2("Square Cutout Glueable", "CONFIGURATIONS", 0, 0, 0, False, 0, Nothing, 0)

        ' Get name of configuration
        swConfiguration = swConfigurationMgr.ActiveConfiguration
        Debug.Print("Configuration name: " & swConfiguration.Name)

        ' Select sketch
        ' Select same dimension in sketch as selected
        ' in previously active configuration
        ' Put the sketch in edit mode
        status = swModelDocExt.SelectByID2("Sketch8", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSketch()
        swModel.ClearSelection2(True)
    
```

```
status = swModelDocExt.SelectByID2("D4@Sketch8@speaker_frame.SLDPRJT", "DIMENSION", -0.00471220094479408, 0.032305394835097, -0.0153009205936774, False, 0, Nothing, 0)
```

```
' Get the selection
```

```
swDisplayDimension = swSelMgr.GetSelectedObject6(1, 0)
```

```
' If selection is not a display dimension, then exit
```

```
If swSelMgr.GetSelectedObjectType3(1, -1) <> swSelectType_e.swSelDIMENSIONS Then Exit Sub
```

```
' Selection is a dimension, so get and print the dimension tolerance
```

```
swDimension = swDisplayDimension.GetDimension2(0)
```

```
swDimensionTolerance = swDimension.Tolerance
```

```
Debug.Print(" Minimum dimension tolerance: " & swDimensionTolerance.GetMinValue)
```

```
Debug.Print(" Maximum dimension tolerance: " & swDimensionTolerance.GetMaxValue)
```

```
' Exit sketch edit mode
```

```
swModel.InsertSketch2(True)
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Change Dimensions of Gear Mate Example (VBA)

This example shows how to change the dimensions of a gear mate.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Selects a face on middle-gear<1> and a face on shaft gear<1>.
' 3. Adds a mate called GearMate1.
' 4. Gets and sets the dimensions of GearMate1.
' 5. Expand MateGroup1 and edit GearMate1.
' 6. Examine the Immediate window to verify that the values shown
'     there match the ratios shown in the GearMate1 PropertyManager page.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swAssembly As SldWorks.AssemblyDoc
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMate As SldWorks.Mate2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swDispDim_1 As SldWorks.DisplayDimension
Dim swDispDim_2 As SldWorks.DisplayDimension
Dim swDim_1 As SldWorks.Dimension
Dim swDim_2 As SldWorks.Dimension
Dim dimValue_1 As Variant
Dim dimValue_2 As Variant
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim mateSelectionMark As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swAssembly = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food processor.sldasm",
swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModel = swAssembly
    Set swModelDocExt = swModel.Extension

    ' Add a gear mate
    mateSelectionMark = 1
    swModel.ViewZoomTo2 0.032214794364982, -2.13641280653619E-03, 2.43727437299652E-02, 7.14815689826596E-02, -2.90907241966369E-02, 2.43727437299654E-02
    status = swModelDocExt.SelectByID2("", "FACE", 5.94157516983955E-02, 1.47722422245806E-02, 1.01883978994124E-02, True, mateSelectionMark, Nothing,
swSelectOption_e.swSelectOptionDefault)
    status = swModelDocExt.SelectByID2("", "FACE", 0.10202121114628, 0.037565135718296, 1.20206517165684E-02, True, mateSelectionMark, Nothing,
swSelectOption_e.swSelectOptionDefault)
    Set swMate = swAssembly.AddMate5(swMateType_e.swMateGEAR, swMateAlign_e.swMateAlignALIGNED, True, 4.79944356756808E-02, 0.001, 0.001, 0.0618, 0.0752,
0.5235987755983, 0.5235987755983, 0.5235987755983, False, False, 0, errors)
    swModel.ClearSelection2 True
    swModel.EditRebuild3

    Set swSelMgr = swModel.SelectionManager
    swModel.ShowFeatureDimensions

    ' Select the gear mate
    status = swModelDocExt.SelectByID2("GearMate1", "MATE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)

    Set swMate = swFeature.GetSpecificFeature2

    ' IMate2::DisplayDimension2 is a property
    ' that requires an index argument
    ' For a gear mate, there are two display dimensions
    ' Thus, IMate2::DisplayDimension2(0) and
    ' IMate2::DisplayDimension2(1) give two display dimensions

    ' Get the gear mates' display dimensions
    Set swDispDim_1 = swMate.DisplayDimension2(0)
    Set swDispDim_2 = swMate.DisplayDimension2(1)

    ' Get the dimensions
    Set swDim_1 = swDispDim_1.GetDimension
    Set swDim_2 = swDispDim_2.GetDimension

    ' Get the value of dimensions
    dimValue_1 = swDim_1.GetSystemValue3(swInConfigurationOpts_e.swThisConfiguration, Empty)
    dimValue_2 = swDim_2.GetSystemValue3(swInConfigurationOpts_e.swThisConfiguration, Empty)
    Debug.Print "Original gear dimensions: " & dimValue_2(0) * 1000 & "mm, " & dimValue_1(0) * 1000 & "mm"

    ' Half the values of dimensions
    dimValue_1(0) = dimValue_1(0) / 2
    dimValue_2(0) = dimValue_2(0) / 2
    errors = swDim_1.SetSystemValue3(dimValue_1(0), swSetValueInConfiguration_e.swSetValue_InThisConfiguration, Empty)
    errors = swDim_2.SetSystemValue3(dimValue_2(0), swSetValueInConfiguration_e.swSetValue_InThisConfiguration, Empty)
    Debug.Print "Updated gear dimensions: " & dimValue_2(0) * 1000 & "mm, " & dimValue_1(0) * 1000 & "mm"

    swModel.EditRebuild3

```

End Sub

Change Direction Reference of Move Face Feature Example (VBA)

This example shows how to change the reference direction of a **Move Face** feature.

```
'-----  
'  
' Preconditions: Part is open that has a Move Face feature.  
'  
' Postconditions: The selected edge becomes the new  
'               direction reference for the Move Face feature.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swMoveFaceFeat As SldWorks.MoveFaceFeatureData  
Dim swEntity As SldWorks.Entity  
Dim boolstatus As Boolean  
Dim dirType As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
  
    ' Get the Move Face feature  
    boolstatus = swModelDocExt.SelectByID2("Move Face1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)  
    swModel.ClearSelection2 True  
    Set swMoveFaceFeat = swFeat.GetDefinition  
  
    ' Roll back the Move Face feature  
    swMoveFaceFeat.AccessSelections swModel, Nothing  
    ' Get the current direction reference  
    Set swEntity = swMoveFaceFeat.GetDirectionReference(dirType)  
    swEntity.Select4 False, Nothing  
    ' Clear the selection  
    swModel.ClearSelection2 True  
  
    ' Select a different edge for the direction reference  
    boolstatus = swModelDocExt.SelectByID2("", "EDGE", -0.05047526142499, 0.03085263679878, 0.04047526142483, False, 0, Nothing, 0)  
    Set swEntity = swSelMgr.GetSelectedObject6(1, 0)  
    swModel.ClearSelection2 True  
    ' Set the direction reference to the just-selected edge  
    swMoveFaceFeat.SetDirectionReference swEntity  
  
    ' Roll back the part with the modified Move Face feature  
    swFeat.ModifyDefinition swMoveFaceFeat, swModel, Nothing  
  
End Sub
```

Change Display Mode to Draft Quality Example (VBA)

This example shows how to change all of the display modes of all of the drawing views in a drawing to draft quality.

```
'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt_assembly.slddrw.
' 2. Move the cursor around in a drawing view to see the
'    current display mode for that drawing view (a blank
'    rectangle indicates that the display mode is high quality).
'
' Postconditions:
' 1. Changes the display modes of all drawing views to draft quality.
' 2. Move the cursor around in a drawing view to see the current display
'    mode for that drawing view (a square with a lighting bolt indicates
'    that the display mode is draft quality).
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSheet As SldWorks.Sheet
    Dim swView As SldWorks.View
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swDraw = swApp.ActiveDoc
    Set swSheet = swDraw.GetCurrentSheet
    Set swView = swDraw.GetFirstView

    While Not swView Is Nothing
        If swSheet.GetName <> swView.Name Then
            ' Does not work on drawing sheet, which IDrawingDoc::GetFirstView returns
            bRet = swView.SetDisplayMode3(False, swView.GetDisplayMode2, True, True): Debug.Assert bRet
        End If
        Set swView = swView.GetNextView
    Wend

End Sub
```

Change Drafting Standard to Custom Drafting Standard (VBA)

This example shows how to change the drafting standard to a custom drafting standard.

```
'-----
' Preconditions: Model document is open and a
'               SOLIDWORKS-supplied drafting standard is set.
'
' Postconditions: Drafting standard is set to the
'               specified custom drafting standard.
'-----
```

Option Explicit

```
Dim swApp           As SldWorks.SldWorks
Dim swModel         As SldWorks.ModelDoc2
Dim swModExt        As SldWorks.ModelDocExtension
Dim bRetVal         As Boolean
Dim sPath           As String
Dim sFileName       As String
Dim vDSNames        As Variant
Dim i               As Integer
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModExt = swModel.Extension
```

```
'Get current SOLIDWORKS-supplied drafting standard
Debug.Print "Current drafting standard..."
Debug.Print " (Standard, NoOptionSpecified) before = " & swModExt.GetUserPreferenceInteger(SwConst.swDetailingDimensionStandard, SwConst.swDetailingNoOptionSpecified)
Debug.Print " (StandardName, NoOptionSpecified) before = " & swModExt.GetUserPreferenceString(SwConst.swDetailingDimensionStandardName, SwConst.swDetailingNoOptionSpecified)
Debug.Print " "
```

```
' Get drafting standard names
' Only the SOLIDWORKS-supplied drafting standards
' are returned; any custom drafting standards are
' not returned
Debug.Print "SOLIDWORKS-supplied drafting standards..."
vDSNames = swModExt.GetDraftingStandardNames
PrintNames vDSNames
Debug.Print " "
```

```
' Load custom drafting standard
bRetVal = swModExt.LoadDraftingStandard("C:\test\MyANSI.sldstd") ' Substitute your custom drafting standard path and filename
```

```
' Get custom drafting standard just-specified
Debug.Print "Standard that custom drafting standard is based on or derived from..."
Debug.Print " (Standard, NoOptionSpecified) after custom loaded = " & swModExt.GetUserPreferenceInteger(SwConst.swDetailingDimensionStandard, SwConst.swDetailingNoOptionSpecified)
Debug.Print " (StandardName, NoOptionSpecified) after custom loaded = " & swModExt.GetUserPreferenceString(SwConst.swDetailingDimensionStandardName, SwConst.swDetailingNoOptionSpecified)
Debug.Print " "
```

```
' Get drafting standard names
' Remember, only the SOLIDWORKS-supplied drafting standards
' are returned; any custom drafting standards are
' not returned
Debug.Print "SOLIDWORKS-supplied drafting standards..."
```

```
vDSNames = swModExt.GetDraftingStandardNames
PrintNames vDSNames
Debug.Print " "

End Sub

Function PrintNames(ByVal vDSNames As Variant)
For i = LBound(vDSNames) To UBound(vDSNames)
    Debug.Print " " & vDSNames(i)
Next i
End Function
```

Change Faces of Move Face Feature Example (VBA)

This example shows how to modify a Move Face feature by changing the defining faces.

```
'-----  
'  
' Preconditions: Part is open that has a Move Face feature.  
'  
' Postconditiions: The selected new face now defines the Move Face  
' feature.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swMoveFaceFeat As SldWorks.MoveFaceFeatureData  
Dim swEnt As SldWorks.Entity  
Dim vFaces As Variant  
Dim pFace As Variant  
Dim vNewFaces As Variant  
Dim pNewFace As Variant  
Dim pNewFaceArr(0) As Object  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
  
    ' Select the Move Face feature  
    boolstatus = swModelDocExt.SelectByID2("Move Face1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)  
    swModel.ClearSelection2 True  
    Set swMoveFaceFeat = swFeat.GetDefinition  
  
    ' Roll back the Move Face feature  
    swMoveFaceFeat.AccessSelections swModel, Nothing  
  
    ' Find and clear the faces that define the Move Face feature  
    vFaces = swMoveFaceFeat.Faces  
    For Each pFace In vFaces  
        Set swEnt = pFace  
        swEnt.Select4 False, Nothing  
        swModel.ClearSelection2 True  
    Next  
  
    ' Select a new face for the Move Face feature  
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.06393265679355, 0.049999999999984, 0.01806458069194, False, 0, Nothing, 0)  
    Set pNewFace = swSelMgr.GetSelectedObject6(1, 0)  
    swModel.ClearSelection2 True  
    Set pNewFaceArr(0) = pNewFace  
    vNewFaces = pNewFaceArr  
    ' Set the new face for the Move Face feature  
    swMoveFaceFeat.Faces = vNewFaces  
  
    ' Modify the Move Face feature and roll back the feature  
    swFeat.ModifyDefinition swMoveFaceFeat, swModel, Nothing  
  
End Sub
```

Change Height of Dome Feature Example (VBA)

This example shows how to change the height of a dome feature.

```
'-----
'
' Preconditions: Dome feature for which a distance was specified is selected.
'
' Postconditions: Height of dome feature is doubled.
'
'-----

Option Explicit
Sub main()
    Dim swApp          As SldWorks.SldWorks
    Dim swPart          As SldWorks.PartDoc
    Dim swModel         As SldWorks.ModelDoc2
    Dim swSelMgr         As SldWorks.SelectionMgr
    Dim swSelData        As SldWorks.SelectData
    Dim swFeat           As SldWorks.feature
    Dim swDome           As SldWorks.DomeFeatureData2
    Dim swFace           As SldWorks.face2
    Dim swEnt            As SldWorks.Entity
    Dim bRet             As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel

    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swDome = swFeat.GetDefinition

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name
    Debug.Print "   Elliptical   = " & swDome.Elliptical
    Debug.Print "   Height       = " & swDome.Height * 1000# & " mm"
    Debug.Print "   ReverseDir   = " & swDome.ReverseDir

    ' Roll back to get access to face
    bRet = swDome.AccessSelections(swModel, Nothing): Debug.Assert bRet

    Set swFace = swDome.Face
    Set swEnt = swFace

    bRet = swEnt.Select4(False, swSelData): Debug.Assert bRet

    ' Make some changes
    swDome.Height = 2# * swDome.Height
    swDome.Elliptical = True

    ' Apply changes
    ' Assert will fire if dome cannot be changed
    bRet = swFeat.ModifyDefinition(swDome, swModel, Nothing): Debug.Assert bRet
End Sub
```

Change Imported File Associated with an Imported Feature Example (VBA)

This example shows how to change the imported file associated with an imported feature.

```
'-----
' Preconditions:
' 1. Verify that c:\temp exists.
' 2. Open public_documents\samples\tutorial\api\cylinder20.sldprt.
' 3. Click File > Save As > Parasolid (*.x_t) in Save as type.
' 4. Type c:\temp before cylinder20.X_T in File name and click Save.
' 5. Open public_documents\samples\tutorial\api\cam roller.sldprt.
' 6. Select 91212_182-P9_SWP_001-1-solid1 in the FeatureManager
'    design tree.
' 7. Open the Immediate window.
'
' Postconditions:
' 1. Replaces the selected imported feature with the specified
'    imported file, c:\temp\cylinder20.x_t.
' 2. Examine the Immediate window and click each imported feature
'    in the FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

Sub main()

```
    Const sImportFileName As String = "C:\temp\cylinder20.x_t"
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swNewModel As SldWorks.ModelDoc2
    Dim swNewPart As SldWorks.PartDoc
    Dim vBodyArr As Variant
    Dim swBody As SldWorks.Body2
    Dim swTempBody As SldWorks.Body2
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "   Feat = " & swFeat.Name
    Debug.Print "   FeatTypeName = " & swFeat.GetTypeName

    ' Open import file into a new part file
    bRet = swApp.LoadFile2(sImportFileName, "")
    Set swNewModel = swApp.ActiveDoc
    Set swNewPart = swNewModel

    ' Only consider solid bodies
    vBodyArr = swNewPart.GetBodies2(swSolidBody, True)

    ' Only consider first solid body
    Set swBody = vBodyArr(0)
    Set swTempBody = swBody.Copy
    bRet = swFeat.SetBody2(swTempBody, False)
    Debug.Print "   Body replaced? " & bRet

    ' Close SOLIDWORKS Design file associated with opening the import file
    swApp.QuitDoc swNewModel.GetTitle

    bRet = swModel.ForceRebuild3(False)
    swModel.ViewZoomtofit2
```

End Sub

Change Labels of SOLIDWORKS Triad Example (VBA)

This example shows how to change the labels of the SOLIDWORKS triad.

```
'-----
'
' Preconditions: SOLIDWORKS Design model document is open.
'
' Postconditions: The labels of the triad are
'                changed and then changed back to
'                their original values.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim rVal As String

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    rVal = swApp.SetUserPreferenceStringValue(swReferenceTriadXLabel, "AltX")
    If rVal = False Then swApp.SendMsgToUser ("ERROR: Unable to change triad label.")

    rVal = swApp.SetUserPreferenceStringValue(swReferenceTriadYLabel, "AltY")
    If rVal = False Then swApp.SendMsgToUser ("ERROR: Unable to change triad label.")

    rVal = swApp.SetUserPreferenceStringValue(swReferenceTriadZLabel, "AltZ")
    If rVal = False Then swApp.SendMsgToUser ("ERROR: Unable to change triad label.")

    swApp.SetUserPreferenceToggle swReferenceTriadUseAlternateLabels, True
    swModel.GraphicsRedraw2

    Stop 'Labels are changed

    swApp.SetUserPreferenceToggle swReferenceTriadUseAlternateLabels, False
    swModel.GraphicsRedraw2

    'Stop 'Labels are changed back to original

End Sub
```

Change Linear Pattern Example (C#)

This example shows how to change a linear pattern from a bodies to a features and faces pattern.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part.
// 2. Creates three boss extrude features.
// 3. Creates a linear pattern using Boss-Extrude2 as a bodies
//    pattern.
// 4. Examine the graphics area and press F5.
// 5. Changes the linear pattern to use Boss-Extrude3 as a
//    features and faces pattern.
// 6. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            FeatureManager swFeatureMgr = default(FeatureManager);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            LinearPatternFeatureData swLinearPatternFeatureData = default(LinearPatternFeatureData);
            object[] sketchSegments = null;
            bool status = false;
            Feature obj = null;
            Feature[] patternFeatures = new Feature[1];

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2015\\templates\\Part.prt", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            //Create boss extrudes
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            sketchSegments = (object[])swSketchMgr.CreateCornerRectangle(0, 0, 0, -0.113876153512535, -0.101331667625686, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeature = (Feature)swFeatureMgr.FeatureExtrusion3(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true, 0, 0, false);
            swSelectionMgr.EnableContourSelection = false;
            status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.105874, -0.015731, 0.0, -0.099776, -0.0152, 0.0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swFeatureMgr.FeatureExtrusion3(true, false, false, 0, 0, 0.01778, 0.00254, false, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, true, true, 0, 0, false);
            swSelectionMgr.EnableContourSelection = false;
            status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            sketchSegments = (object[])swSketchMgr.CreateCornerRectangle(-0.10892213539114, -0.0783168275860362, 0, -0.0879628279544704, -0.0928855015339991, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeature = (Feature)swFeatureMgr.FeatureExtrusion3(true, false, false, 0, 0, 0.01778, 0.01778, false, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, true, true, 0, 0, false);
            swSelectionMgr.EnableContourSelection = false;

            //Create linear pattern using Boss-Extrude2 as bodies pattern
            status = swModelDocExt.SelectByID("", "EDGE", -0.091185205959107, -2.85588595829722E-05, 0.00255940246768205, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Boss-Extrude2", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("", "EDGE", -0.091185205959107, -2.85588595829722E-05, 0.00255940246768205, true, 1, null, 0);
            status = swModelDocExt.SelectByID("Boss-Extrude2", "SOLIDBODY", 0, 0, 0, true, 256, null, 0);
            swFeature = (Feature)swFeatureMgr.FeatureLinearPattern4(3, 0.0254, 1, 0.00254, true, false, "NULL", "NULL", false, false, false, false, false, true, true, false, false, 0, 0);

            System.Diagnostics.Debugger.Break();
        }
    }
}
```

```

//Examine the graphics area
//Press F5

//Select LPattern1
//Get whether LPattern1 is a features and faces pattern or a bodies pattern
status = swModelDocExt.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
swLinearPatternFeatureData = (LinearPatternFeatureData)swFeature.GetDefinition();
swLinearPatternFeatureData.AccessSelections(swModel, null);
Debug.Print("Original LPattern1 is a features and faces pattern (true if a
bodies pattern)? " + swLinearPatternFeatureData.BodyPattern);

//Change LPattern1 to features and faces pattern
status = swModelDocExt.SelectByID2("Boss-Extrude3", "BODYFEATURE", 0, 0, 0, true, 0, null, 0);
obj = (Feature)swSelectionMgr.GetSelectedObject6(1, 0);
patternFeatures[0] = (Feature)obj;
DispatchWrapper[] patternFeaturesArray = null;
patternFeaturesArray = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray(patternFeatures);
swLinearPatternFeatureData.BodyPattern = false;
swLinearPatternFeatureData.PatternFeatureArray = patternFeaturesArray;

swFeature.ModifyDefinition(swLinearPatternFeatureData, swModel, null);

//Get whether LPattern1 is a features and faces pattern or a bodies pattern
Debug.Print("Modified LPattern1 is a features and faces pattern (false if a
features and faces pattern)? " + swLinearPatternFeatureData.BodyPattern);
}

public DispatchWrapper[] ObjectArrayToDispatchWrapperArray(object[] Objects)
{
    int ArraySize = 0;
    ArraySize = Objects.GetUpperBound(0);
    DispatchWrapper[] d = new DispatchWrapper[ArraySize + 1];
    int ArrayIndex = 0;
    for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
    {
        d[ArrayIndex] = new DispatchWrapper(Objects[ArrayIndex]);
    }
    return d;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Change Linear Pattern Example (VBA)

This example shows how to change a linear pattern from a bodies to a features and faces pattern.

```

'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates three boss extrude features.
' 3. Creates a linear pattern using Boss-Extrude2 as a bodies
'    pattern.
' 4. Examine the graphics area and press F5.
' 5. Changes the linear pattern to use Boss-Extrude3 as a
'    features and faces pattern.
' 6. Examine the Immediate window and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swLinearPatternFeatureData As SldWorks.LinearPatternFeatureData
Dim sketchSegments As Variant
Dim status As Boolean
Dim obj As Object
Dim patternFeatures(0) As Object

Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchMgr = swModel.SketchManager
    Set swFeatureMgr = swModel.FeatureManager
    Set swSelectionMgr = swModel.SelectionManager

    'Create boss extrudes
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    sketchSegments = swSketchMgr.CreateCornerRectangle(0, 0, 0, -0.113876153512535, -0.101331667625686, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)
    swSelectionMgr.EnableContourSelection = False
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swSketchSegment = swSketchMgr.CreateCircle(-0.105874, -0.015731, 0#, -0.099776, -0.0152, 0#)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.01778, 0.00254, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, True, True, 0, 0, False)
    swSelectionMgr.EnableContourSelection = False
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    sketchSegments = swSketchMgr.CreateCornerRectangle(-0.10892213539114, -7.83168275860362E-02, 0, -8.79628279544704E-02, -9.28855015339991E-02, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.01778, 0.01778, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, True, True, 0, 0, False)
    swSelectionMgr.EnableContourSelection = False

    'Create linear pattern using Boss-Extrude2 as bodies pattern
    status = swModelDocExt.SelectByID2("", "EDGE", -0.091185205959107, -2.85588595829722E-05, 2.55940246768205E-03, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Boss-Extrude2", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("", "EDGE", -0.091185205959107, -2.85588595829722E-05, 2.55940246768205E-03, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Boss-Extrude2", "SOLIDBODY", 0, 0, 0, True, 256, Nothing, 0)
    Set swFeature = swFeatureMgr.FeatureLinearPattern4(3, 0.0254, 1, 0.00254, True, False, "NULL", "NULL", False, False, False, False, True,
True, False, False, 0, 0)

    Stop
    'Examine the graphics area
    'Press F5

```

```

'Select LPattern1
'Get whether LPattern1 is a features and faces pattern or a bodies pattern
status = swModelDocExt.SelectByID("LPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
Set swLinearPatternFeatureData = swFeature.GetDefinition
swLinearPatternFeatureData.AccessSelections swModel, Nothing
Debug.Print "Original LPattern1 is a features and faces pattern or a bodies pattern (true if a bodies pattern)? " &
swLinearPatternFeatureData.BodyPattern

'Change LPattern1 to features and faces pattern
status = swModelDocExt.SelectByID("Boss-Extrude3", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
Set obj = swSelectionMgr.GetSelectedObject6(1, 0)
Set patternFeatures(0) = obj
swLinearPatternFeatureData.BodyPattern = False
swLinearPatternFeatureData.PatternFeatureArray = patternFeatures

swFeature.ModifyDefinition swLinearPatternFeatureData, swModel, Nothing

'Get whether LPattern1 is a features and faces pattern or a bodies pattern
Debug.Print "Modified LPattern1 is a features and faces pattern or a bodies pattern (false if a features and faces pattern)? " &
swLinearPatternFeatureData.BodyPattern

End Sub

```

Change Linear Pattern Example (VB.NET)

This example shows how to change a linear pattern from a bodies to a features and faces pattern.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
-----
Postconditions:
1. Opens a new part.
2. Creates three boss extrude features.
3. Creates a linear pattern using Boss-Extrude2 as a bodies pattern.
4. Examine the graphics area and press F5.
5. Changes the linear pattern to use Boss-Extrude3 as a features and faces pattern.
6. Examine the Immediate window and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swFeatureMgr As FeatureManager
        Dim swSelectionMgr As SelectionMgr
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swLinearPatternFeatureData As LinearPatternFeatureData
        Dim sketchSegments As Object
        Dim status As Boolean
        Dim obj As Object
        Dim patternFeatures(0) As Object

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension
        swSketchMgr = swModel.SketchManager
        swFeatureMgr = swModel.FeatureManager
        swSelectionMgr = swModel.SelectionManager

        'Create boss extrudes
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOption
Specified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOp
tionSpecified, True)
        sketchSegments = swSketchMgr.CreateCornerRectangle(0, 0, 0, -0.113876153512535, -0.101331667625686, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.0174532925199433, 0.017453292
5199433, False, False, False, True, True, 0, 0, False)
        swSelectionMgr.EnableContourSelection = False
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchMgr.CreateCircle(-0.105874, -0.015731, 0.0#, -0.099776, -0.0152, 0.0#)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.01778, 0.00254, False, False, False, False, 0.0174532925199433, 0.017453292
5199433, False, False, False, True, True, 0, 0, False)
        swSelectionMgr.EnableContourSelection = False
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOption
Specified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOp
tionSpecified, True)
        sketchSegments = swSketchMgr.CreateCornerRectangle(-0.10892213539114, -0.0783168275860362, 0, -0.0879628279544704, -0.0928855015339991, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.01778, 0.01778, False, False, False, False, 0.0174532925199433, 0.017453292
5199433, False, False, False, True, True, 0, 0, False)
        swSelectionMgr.EnableContourSelection = False

        'Create linear pattern using Boss-Extrude2 as bodies pattern
        status = swModelDocExt.SelectByID2("", "EDGE", -0.091185205959107, -0.0000285588595829722, 0.00255940246768205, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Boss-Extrude2", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("", "EDGE", -0.091185205959107, -0.0000285588595829722, 0.00255940246768205, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Boss-Extrude2", "SOLIDBODY", 0, 0, 0, True, 256, Nothing, 0)
        swFeature = swFeatureMgr.FeatureLinearPattern4(3, 0.0254, 1, 0.00254, True, False, "NULL", "NULL", False, False, False, False, False, False, True,
True, False, False, 0, 0)

        Stop
        'Examine the graphics area
        'Press F5

        'Select LPattern1
        'Get whether LPattern1 is a features and faces pattern or a bodies pattern
        status = swModelDocExt.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swLinearPatternFeatureData = swFeature.GetDefinition
        swLinearPatternFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Original LPattern1 is a features and faces pattern or a bodies pattern (true if a
bodies pattern)? " & swLinearPatternFeatureData.BodyPattern)

        'Change LPattern1 to features and faces pattern
        status = swModelDocExt.SelectByID2("Boss-Extrude3", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
    
```

```

obj = swSelectionMgr.GetSelectedObject6(1, 0)
patternFeatures(0) = obj
Dim patternFeaturesArray() As DispatchWrapper
patternFeaturesArray = ObjectArrayToDispatchWrapperArray(patternFeatures)
swLinearPatternFeatureData.BodyPattern = False
swLinearPatternFeatureData.PatternFeatureArray = patternFeaturesArray
swFeature.ModifyDefinition(swLinearPatternFeatureData, swModel, Nothing)

'Get whether LPattern1 is a features and faces pattern or a bodies pattern
Debug.Print("Modified LPattern1 is a features and faces pattern or a bodies pattern (false if a
features and faces pattern)? " & swLinearPatternFeatureData.BodyPattern)

End Sub

Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As DispatchWrapper
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Change Material Properties Example (VBA)

This example shows how to change the color of the selected assembly component.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\landing_gear.sldasm.
' 2. Click oleostrut<1> in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Changes the color of the selected component to blue.
' 2. Examine the component in the graphics area and
'    the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not save
'       changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swCompModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp As SldWorks.Component2
    Dim vMatProp As Variant
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent2(1)

    vMatProp = swComp.MaterialPropertyValues
    If IsEmpty(vMatProp) Then
        ' Empty if no component-level colors specified,
        ' so get from underlying model
        Set swCompModel = swComp.GetModelDoc
        If swCompModel Is Nothing Then
            ' Component is lightweight
            Debug.Print "Selected component is lightweight; exiting macro."
            Exit Sub
        End If
        vMatProp = swCompModel.MaterialPropertyValues
    End If

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "  Component = " & swComp.Name2 & " (" & swComp.ReferencedConfiguration & ")"
    Debug.Print "    State = " & swComp.GetSuppression
    Debug.Print "    Suppressed = " & swComp.IsSuppressed
    Debug.Print "    Hidden = " & swComp.IsHidden(False)
    Debug.Print "    RGB = [" & vMatProp(0) * 255# & ", " & vMatProp(1) * 255# & ", " & vMatProp(2) * 255# & "]"
    Debug.Print "    Ambient = " & vMatProp(3)
    Debug.Print "    Diffuse = " & vMatProp(4)
    Debug.Print "    Specular = " & vMatProp(5)
    Debug.Print "    Shininess = " & vMatProp(6)
    Debug.Print "    Transparency = " & vMatProp(7)
    Debug.Print "    Emission = " & vMatProp(8)

    ' Force component color to blue
    vMatProp(0) = 0#
    vMatProp(1) = 0#
    vMatProp(2) = 1#
    swComp.MaterialPropertyValues = vMatProp

    ' Deselect component to see new color
    swModel.ClearSelection2 True

End Sub

```

Change_Name_of_Component_Example_CSharp (Topic)

SOLIDWORKS API Help

Change Name of Component Example (C#)

This example shows how to change the name of a component.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Select a component in the assembly.
// 3. Open the Immediate window.
// 4. Press F5.
//
// Postconditions:
// 1. The selected component's name is
//    changed to SW.
// 2. Examine the Immediate window and
//    FeatureManager design tree to verify.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ComponentName2CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swComp = default(Component2);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent3(1, 0);
            if (swComp == null)
            {
                Debug.Print("Select a component and run the macro again.");
                return;
            }
            else
            {
                // swUserPreferenceToggle_e.swExtRefUpdateCompNames must be set to
                // false to change the name of a component using IComponent2::Name2
                swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swExtRefUpdateCompNames, false);

                // Print original name of component
                Debug.Print(" Original name of component = " + swComp.Name2);

                // Change name of component
                swComp.Name2 = "SW";

                // Print new name of component
                Debug.Print(" New name of component    = " + swComp.Name2);
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}
```

Change Name of Component Example (VBA)

This example shows how to change the name of a component.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Select a component in the assembly.
' 3. Open the Immediate window.
' 4. Run the macro.
'
' Postconditions:
' 1. The selected component's name is
'    changed to SW.
' 2. Examine the Immediate window and
'    FeatureManager design tree to verify.
'-----
Option Explicit

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swComp              As SldWorks.Component2

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)
    If swComp Is Nothing Then
        Debug.Print "Select a component and run the macro again."
        Exit Sub
    Else

        ' swUserPreferenceToggle_e.swExtRefUpdateCompNames must be set to
        ' False to change the name of a component using IComponent2::Name2
        swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swExtRefUpdateCompNames, False

        ' Print original name of component
        Debug.Print (" Original name of component = " & swComp.Name2)

        ' Change name of component
        swComp.Name2 = "SW"

        ' Print new name of component
        Debug.Print (" New name of component      = " & swComp.Name2)
    End If
End Sub
```

Change Name of Component Example (VB.NET)

This example shows how to change the name of a component.

```
.....
' Preconditions:
' 1. Open an assembly document.
' 2. Select a component in the assembly.
' 3. Open the Immediate window.
' 4. Press F5.
'
' Postconditions:
' 1. The selected component's name is
'   changed to "SW".
' 2. Examine the Immediate window and
'   FeatureManager design tree to verify.
.....
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swComp As Component2

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)
        If swComp Is Nothing Then
            Debug.Print("Select a component and run the macro again.")
            Exit Sub
        Else

            ' swUserPreferenceToggle_e.swExtRefUpdateCompNames must be set to
            ' False to change the name of a component using IComponent2::Name2
            swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swExtRefUpdateCompNames, False)

            ' Print original name of component
            Debug.Print(" Original name of component = " & swComp.Name2)

            ' Change name of component
            swComp.Name2 = "SW"

            ' Print new name of component
            Debug.Print(" New name of component    = " & swComp.Name2)
        End If
    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Change Note Text Example (VBA)

This example shows how to change the note on a drawing template by opening the drawing template for editing and then selecting the note and using the INote::SetText method to modify the note.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\2012-sm.slddrw.
'
' Postconditions:
' 1. Examine the title block in the drawing.
' 2. Gets the first drawing view, which is the drawing template.
' 3. Iterates all notes in the drawing template, replacing notes
'    matching initString with newString.
' 4. Examine the title block again to verify step 3.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----

Option Explicit

Const NumStrings As Long = 3
Dim initString(NumStrings) As String
Dim newString(NumStrings) As String

Private Sub InitStrings()
    ' Initial strings from drawing
    initString(0) = "2012-sm"
    initString(1) = "WEIGHT:"
    initString(2) = "FINISH:"

    ' Strings to replace initial strings in drawing
    newString(0) = "NEW 2012-sm"
    newString(1) = "NEW WEIGHT:"
    newString(2) = "NEW FINISH:"
End Sub

Private Sub DoReplaceString(ByRef sNoteText As String)
    Dim i As Long

    For i = 0 To NumStrings
        sNoteText = Replace(sNoteText, initString(i), newString(i), 1, -1, vbTextCompare)
    Next i
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim swNote As SldWorks.Note
    Dim sNoteText As String
    Dim nTextCount As Long
    Dim i As Long

    InitStrings

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swView = swDraw.GetFirstView ' This is the drawing template

    While Not swView Is Nothing
        Set swNote = swView.GetFirstNote2
        While Not swNote Is Nothing
            If swNote.IsCompoundNote Then
                nTextCount = swNote.GetTextCount
                For i = 1 To nTextCount
                    sNoteText = swNote.GetTextAtIndex(i)
                    DoReplaceString sNoteText
                    swNote.SetTextAtIndex i, sNoteText
                Next i
            Else
                sNoteText = swNote.GetText
                DoReplaceString sNoteText
                swNote.SetText sNoteText
            End If
            Set swNote = swNote.GetNext
        Wend
        Set swView = swView.GetNextView
    Wend

End Sub

```

Change Number of Copies for Move/Copy Example (VBA)

This example shows how to change the number of copies for a move/copy.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a solid body and two vertices.
' 3. Inserts a move/copy body feature.
' 4. Modifies the move/copy body (increases number of copies
'    from 1 to 4)
' 5. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swMoveCopyBodyFeatureData As SldWorks.MoveCopyBodyFeatureData
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swEdge As Edge
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim copyStatus As Boolean
Dim number As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager

    'Select solid body and vertices for move/copy body feature
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)

    'Insert move/copy body feature
    Set swFeature = swFeatureManager.InsertMoveCopyBody2(0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, True, 1)

    'Roll forward the feature and get and set move/copy body feature data
    Set swMoveCopyBodyFeatureData = swFeature.GetDefinition
    status = swMoveCopyBodyFeatureData.AccessSelections(swModel, Nothing)

    copyStatus = swMoveCopyBodyFeatureData.Copy
    Debug.Print "Can copy? " & copyStatus
    number = swMoveCopyBodyFeatureData.NumberOfCopies
    Debug.Print "Number of copies of move/copy body before copy: " & number
    If number < 4 Then
        swMoveCopyBodyFeatureData.NumberOfCopies = 4
    End If
    number = swMoveCopyBodyFeatureData.NumberOfCopies
    Debug.Print "Number of copies of move/copy body after copy: " & number

    'Roll back
    status = swFeature.ModifyDefinition(swMoveCopyBodyFeatureData, swModel, Nothing)
    swMoveCopyBodyFeatureData.ReleaseSelectionAccess

    swModel.ViewZoomtofit2

End Sub

```

Change Pitch of Helix Example (C#)

This example shows how to change the pitch of a helix.

```
//-----
// Preconditions:
// 1. Open a part containing a helix feature.
// 2. Select the helix feature.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Modifies the pitch of selected helix feature.
// 2. Gets the name of helix feature, original pitch, and
//    modified pitch values.
// 3. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace PitchHelixFeatureDataCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            HelixFeatureData swHelix = default(HelixFeatureData);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swHelix = (HelixFeatureData)swFeat.GetDefinition();

            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print(" Original pitch = " + swHelix.Pitch * 1000.0 + " mm");

            // Change the pitch value
            swHelix.Pitch = 1.25 * swHelix.Pitch;
            Debug.Print(" Modified pitch = " + swHelix.Pitch * 1000.0 + " mm");

            // Apply the change
            bRet = swFeat.ModifyDefinition(swHelix, swModel, null);

            Debug.Assert(bRet);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Change Pitch of Helix Example (VBA)

This example shows how to change the pitch of a helix.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a helix feature.  
' 2. Select the helix feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Modifies the pitch of selected helix feature.  
' 2. Gets the name of helix feature, original pitch, and  
'    modified pitch values.  
' 3. Examine the Immediate window and graphics area.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim swHelix As SldWorks.HelixFeatureData  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swHelix = swFeat.GetDefinition  
  
    Debug.Print "Feature = " & swFeat.Name  
    Debug.Print " Original pitch = " & swHelix.Pitch * 1000# & " mm"  
  
    ' Change the pitch value  
    swHelix.Pitch = 1.25 * swHelix.Pitch  
    Debug.Print " Modified pitch = " & swHelix.Pitch * 1000# & " mm"  
  
    ' Apply the change  
    bRet = swFeat.ModifyDefinition(swHelix, swModel, Nothing): Debug.Assert bRet  
  
End Sub
```

Change Pitch of Helix Example (VB.NET)

This example shows how to change the pitch of a helix.

```
'-----
' Preconditions:
' 1. Open a part that contains a helix feature.
' 2. Select the helix feature.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Modifies the pitch of selected helix feature.
' 2. Gets the name of helix feature, original pitch, and
'    modified pitch values.
' 3. Examine the Immediate window and graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swHelix As HelixFeatureData
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swHelix = swFeat.GetDefinition

        Debug.Print("Feature = " & swFeat.Name)
        Debug.Print("  Original pitch = " & swHelix.Pitch * 1000.0# & " mm")

        ' Change the pitch value
        swHelix.Pitch = 1.25 * swHelix.Pitch
        Debug.Print("  Modified pitch = " & swHelix.Pitch * 1000.0# & " mm")

        ' Apply the change
        bRet = swFeat.ModifyDefinition(swHelix, swModel, Nothing) : Debug.Assert(bRet)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>

    Public swApp As SldWorks

End Class
```

Change Radial to Diametric Style Example (VBA)

This example shows how to change radial style to diametric style.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Right-click Annotations and click Show Feature Dimensions.  
'  
' Postconditions:  
' 1. Selects a radial dimension.  
' 2. Changes the selected radial dimension to a diametric  
'    dimension.  
' 3. Examine the graphics area.  
'  
' NOTE: Because the part is used elsewhere, do not save  
' changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swDispDim As SldWorks.DisplayDimension  
    Dim boolstatus As Boolean  
    Dim longstatus As Long  
    Dim longwarnigs As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    boolstatus = swModelDocExt.SelectByID("D1@Sketch2@block.SLDPRT", "DIMENSION", 1.6984805113666E-02, -0.028515153676096, -3.13572600146445E-04, False,  
0, Nothing, 0)  
    Set swDispDim = swSelMgr.GetSelectedObject6(1, -1)  
  
    ' Toggle between radial and diametric styles  
    If swDispDim.Diametric Then  
        swDispDim.Diametric = False  
    Else  
        swDispDim.Diametric = True  
    End If  
  
    ' Redraw to see changes  
    swModel.GraphicsRedraw2  
  
End Sub
```

Change Reference Dimension Example (VBA)

This example shows how to change a reference dimension in a drawing without changing the model.

```
'-----  
'  
' Preconditions:  
'      (1) Drawing document is open.  
'      (2) Reference dimension named RD1@Drawing View1 exists in the drawing.  
'  
' Postconditions: Value of RD1@Drawing View1 changed to 10.0, but model is not changed.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swDispDim As SldWorks.DisplayDimension  
Dim boolstatus As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swModelDocExt = swModel.Extension  
Set swSelMgr = swModel.SelectionManager  
boolstatus = swModelDocExt.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.2765561531303, 0.1334812822687, 0, False, 0, Nothing, 0)  
  
Set swDispDim = swSelMgr.GetSelectedObject6(1, 0)  
swDispDim.SetText SwConst.swDimensionTextParts_e.swDimensionTextAll, "10.0"  
  
End Sub
```

Change Referenced Configuration Example (VBA)

This example shows how to change the configuration of a component in an assembly without opening the part.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Select a component.  
'  
' Postconditions:  
' 1. Changes the referenced configuration.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swModel As SldWorks.ModelDoc  
    Dim swComp As SldWorks.Component  
    Dim RefCfg As String  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swAssy = swModel  
    Set swSelMgr = swModel.SelectionManager  
  
    Set swComp = swSelMgr.GetSelectedObject6(1, -1)  
  
    ' Change component configuration to "in-use"  
    swComp.ReferencedConfiguration = ""  
    RefCfg = swComp.ReferencedConfiguration  
    Debug.Print "Referenced configuration: " & RefCfg  
  
    ' For changes to take effect  
    swModel.EditRebuild3  
  
End Sub
```

Change the Plane of a Sketch Example (VBA)

This example shows how to modify the plane of a sketch.

```
'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Creates a new part document with a sketch of a spline.
' 2. Changes the plane of the sketch Top Plane to the Front Plane.
' 3. Examine the FeatureManager design tree and graphics area.
'-----
```

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim skSegment As SldWorks.SketchSegment
Dim boolstatus As Boolean
```

Option Explicit

Sub main()

```
Set swApp = Application.SldWorks
Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
```

```
boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -4.94443883882606E-02, 0.010829578664819, 1.87336739521956E-02,
True, 0, Nothing, 0)
Part.SketchManager.InsertSketch True
```

```
Dim pointArray As Variant
Dim points() As Double
ReDim points(0 To 11) As Double
points(0) = -6.96700449874595E-02
points(1) = -2.05096087491173E-02
points(2) = 0
points(3) = -3.49133034431539E-02
points(4) = 1.51865041882777E-02
points(5) = 0
points(6) = 1.83177421652422E-02
points(7) = 0
points(8) = 0
points(9) = 0.060902578651959
points(10) = 3.36608082523681E-02
points(11) = 0
pointArray = points
```

```
Set skSegment = Part.SketchManager.CreateSpline((pointArray))
Part.SketchManager.InsertSketch True
```

```
boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = Part.DeSelectByID("Top Plane", "PLANE", 0, 0, 0)
```

```
' Select sketch and new plane for the sketch
boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
' Change the plane of the sketch
boolstatus = Part.Extension.ChangeSketchPlane(1, Nothing)
boolstatus = Part.EditRebuild3()
```

```
Part.ShowNamedView2 "*Isometric", 7
boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
```

End Sub

Change the Plane of a Sketch Example (C#)

This example shows how to modify the plane of a sketch.

```
//-----
// Preconditions: Verify that the specified template exists.
//
// Postconditions:
// 1. Creates a new part document with a sketch of a spline.
// 2. Changes the plane of the sketch Top Plane to the Front Plane.
// 3. Examine the FeatureManager design tree and graphics area.
//-----

using Microsoft.VisualBasic;

using System;

using System.Collections;

using System.Collections.Generic;

using System.Data;

using System.Diagnostics;

using SolidWorks.Interop.sldworks;

using SolidWorks.Interop.swconst;

using System.Runtime.InteropServices;

namespace ChangeSketchPlane_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;

        SketchSegment skSegment;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS
2017\\templates\\Part.prtdot", 0, 0, 0);

            boolstatus = Part.Extension.SelectByID2("Top
Plane", "PLANE", -0.0494443883882606,
0.010829578664819, 0.0187336739521956, true,
0, null,
0);

            Part.SketchManager.InsertSketch(true);

            object pointArray = null;

            double[]
points = new double[12];

            points[0] = -0.0696700449874595;

            points[1] = -0.0205096087491173;

            points[2] = 0;

            points[3] = -0.0349133034431539;

            points[4] = 0.0151865041882777;

            points[5] = 0;

            points[6] = 0.0183177421652422;

            points[7] = 0;

            points[8] = 0;

            points[9] = 0.060902578651959;

            points[10] = 0.0336608082523681;

            points[11] = 0;

            pointArray = points;
```

```

        skSegment = Part.SketchManager.CreateSpline((pointArray));

        Part.SketchManager.InsertSketch(true);

        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH",
0, 0, 0, false,
0, null,
0);

        boolstatus = Part.Extension.SelectByID2("Top
Plane", "PLANE", 0, 0, 0, true, 0, null, 0);

        boolstatus = Part.DeSelectByID("Top Plane", "PLANE", 0, 0, 0);

        // Select sketch and new plane
for the sketch

        boolstatus = Part.Extension.SelectByID2("Front
Plane", "PLANE", 0, 0, 0, true, 0, null, 0);

        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH",
0, 0, 0, false,
0, null,
0);

        // Change the plane of the
sketch

        boolstatus = Part.Extension.ChangeSketchPlane(1, null);

        boolstatus = Part.EditRebuild3();

        Part.ShowNamedView2("**Isometric",
7);

        boolstatus = Part.Extension.SelectByID2("Front
Plane", "PLANE", 0, 0, 0, true, 0, null, 0);

    }

    public SldWorks swApp;

}

}

```

Change the Plane of a Sketch Example (VB.NET)

This example shows how to modify the plane of a sketch.

```
'-----
' Preconditions: Verify that the specified template exists.
```

```
' Postconditions:
' 1. Creates a new part document with a sketch of a spline.
' 2. Changes the plane of the sketch Top Plane to the Front Plane.
' 3. Examine the FeatureManager design tree and graphics area.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim skSegment As SketchSegment
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -0.0494443883882606, 0.010829578664819, 0.0187336739521956,
True, 0, Nothing, 0)
        Part.SketchManager.InsertSketch(True)
```

```
        Dim pointArray As Object
        Dim points(11) As Double
```

```
        points(0) = -0.0696700449874595
        points(1) = -0.0205096087491173
        points(2) = 0
        points(3) = -0.0349133034431539
        points(4) = 0.0151865041882777
        points(5) = 0
        points(6) = 0.0183177421652422
        points(7) = 0
        points(8) = 0
        points(9) = 0.060902578651959
        points(10) = 0.0336608082523681
        points(11) = 0
        pointArray = points
```

```
        skSegment = Part.SketchManager.CreateSpline((pointArray))
        Part.SketchManager.InsertSketch(True)
```

```
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = Part.DeSelectByID("Top Plane", "PLANE", 0, 0, 0)
```

```
        ' Select sketch and new plane for the sketch
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        ' Change the plane of the sketch
        boolstatus = Part.Extension.ChangeSketchPlane(1, Nothing)
        boolstatus = Part.EditRebuild3()
```

```
        Part.ShowNamedView2("**Isometric", 7)
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Change Sketch Plane Example (VBA)

This example shows how to change which plane a sketch is on.

```
'-----  
' Preconditions: Open a part document that  
' contains:  
' * Sketch1 sketched on the Front Plane.  
' * Plane named Plane1.  
'  
' Postconditions:  
' 1. Moves Sketch1 to Plane1.  
' 2. Examine the graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim vConfigNames As Variant  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
  
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)  
    If (1) Then  
        boolstatus = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, True, 0, Nothing, 0)  
    End If  
    If (0) Then  
        boolstatus = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)  
    End If  
  
    vConfigNames = swModel.GetConfigurationNames()  
    boolstatus = swModelDocExt.ChangeSketchPlane(swThisConfiguration, vConfigNames(0))  
  
    boolstatus = swModel.EditRebuild3()  
End Sub
```

Change Tags in Hole Table Example (C#)

This example shows how to change the tags in a hole table.

```
//-----
// Preconditions:
// 1. Open a drawing document that contains
//    a hole table named Hole Table1 that has
//    has four columns (TAG, X LOC, Y LOC, and SIZE)
//    and at least five rows.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Disables updating hole table tags.
// 2. Changes tag in column 1, row 2 to Test1.
// 3. Changes tag in column 1, row 5 to Test2.
// 4. Enables updating of hole table and model view
//    and shows new tags.
// 5. Examine the Immediate window and hole table.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace RenameHoleTableTag_CSharp.csproj
{
    partial
    class SolidWorksMacro
    {
        public
        void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            HoleTable swHoleTable = default(HoleTable);

            bool boolstatus = false;
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            boolstatus = swModel.Extension.SelectByID2("Hole Table1", "HOLETABLE",
            0, 0, 0, false, 0, null, 0);
            swHoleTable = (HoleTable)swSelMgr.GetSelectedObject6(1, -1);
            swHoleTable.EnableUpdate = false;

            //Disable
            updating hole table tags
            Debug.Print("Original name of tag in column1, row 2: " + swHoleTable.get_HoleTag(1));

            swHoleTable.set_HoleTag(1, "Test1");
            Debug.Print("New name of tag in column1, row 2: " + swHoleTable.get_HoleTag(1));
            Debug.Print("");
            Debug.Print("Original name of tag in column1, row 5: " + swHoleTable.get_HoleTag(4));
            swHoleTable.set_HoleTag(4, "Test1");
            //
            Fails because same name is used in row 2
            swHoleTable.set_HoleTag(4, "Test2");
            Debug.Print("New name of tag in column1, row 2: " + swHoleTable.get_HoleTag(4));

            //Enable
            updating hole table tags
            swHoleTable.EnableUpdate = true;
        }
        public SldWorks swApp;
    }
}
```

Change Tags in Hole Table Example (VBA)

This example shows how to change the tags in a hole table.

```

'-----
' Preconditions:
' 1. Open a drawing document that contains
'    a hole table named Hole Table1 that has
'    has four columns (TAG, X LOC, Y LOC, and SIZE)
'    and at least five rows.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Disables updating hole table tags.
' 2. Changes tag in column 1, row 2 to Test1.
' 3. Changes tag in column 1, row 5 to Test2.
' 4. Enables updating of hole table and model view
'    and shows new tags.
' 5. Examine the Immediate window and hole table.
'-----
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swHoleTable As SldWorks.HoleTable
    Dim boolstatus As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    boolstatus = swModel.Extension.SelectByID("Hole Table1", "HOLETABLE", 0, 0, 0, False, 0, Nothing, 0)
    Set swHoleTable = swSelMgr.GetSelectedObject6(1, -1)

    swHoleTable.EnableUpdate = False ' Don't update hole table or model view

    ' Change hole tags
    Debug.Print "Original name of tag in column1, row 2:  " & swHoleTable.HoleTag(1)
    swHoleTable.HoleTag(1) = "Test1"
    Debug.Print "New name of tag in column1, row 2:      " & swHoleTable.HoleTag(1)
    Debug.Print ""
    Debug.Print "Original name of tag in column1, row 5:  " & swHoleTable.HoleTag(4)
    swHoleTable.HoleTag(4) = "Test1" ' Fails because same name is used in row 2
    swHoleTable.HoleTag(4) = "Test2"
    Debug.Print "New name of tag in column1, row 2:      " & swHoleTable.HoleTag(4)

    swHoleTable.EnableUpdate = True ' Update hole table and model view

End Sub

```

Change Tags in Hole Table Example (VB.NET)

This example shows how to change the tags in a hole table.

```

'-----
' Preconditions:
' 1. Open a drawing document that contains
'    a hole table named Hole Table1 that has
'    has four columns (TAG, X LOC, Y LOC, and SIZE)
'    and at least five rows.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Disables updating hole table tags.
' 2. Changes tag in column 1, row 2 to Test1.
' 3. Changes tag in column 1, row 5 to Test2.
' 4. Enables updating of hole table and model view
'    and shows new tags.
' 5. Examine the Immediate window and hole table.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public
    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swHoleTable As HoleTable
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        boolstatus = swModel.Extension.SelectByID2("Hole
Table1", "HOLETABLE", 0, 0, 0, False, 0, Nothing, 0)
        swHoleTable = swSelMgr.GetSelectedObject6(1,
-1)

        swHoleTable.EnableUpdate = False 'Disable updating
hole table tags

        Debug.Print("Original name of tag in column1, row 2:  "
& swHoleTable.HoleTag(1))
        swHoleTable.HoleTag(1) = "Test1"
        Debug.Print("New name of tag in column1, row 2:      "
& swHoleTable.HoleTag(1))
        Debug.Print("")
        Debug.Print("Original name of tag in column1, row 5:  "
& swHoleTable.HoleTag(4))
        swHoleTable.HoleTag(4) = "Test1" '
Fails because same name is used in row 2
        swHoleTable.HoleTag(4) = "Test2"
        Debug.Print("New name of tag in column1, row 2:      "
& swHoleTable.HoleTag(4))

        swHoleTable.EnableUpdate = True 'Enable updating
hole table tags
    End
    Sub

        ''' <summary>
        ''' The SldWorks swApp variable is pre-assigned for you.
        ''' </summary>
        Public swApp As SldWorks

End Class

```

Change Text Format Example (VBA)

This example shows how to change the text format of an annotation.

```
'-----
' Preconditions:
' 1. Open a part, assembly, or drawing that has at least
'   one annotation containing text.
' 2. Select that annotation.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Changes the text format of the selected annotation.
' 2. Examine the annotation in the graphics area and
'   the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swAnnObj             As Object
    Dim swAnn                As SldWorks.Annotation
    Dim swTextFormat         As SldWorks.TextFormat
    Dim i                   As Long
    Dim bRet                 As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swAnnObj = swSelMgr.GetSelectedObject6(1, -1)
    Set swAnn = swAnnObj.GetAnnotation

    'Get text format information
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swAnn.GetName & " <" & swAnn.GetType & ">"

    For i = 0 To swAnn.GetTextFormatCount - 1
        Set swTextFormat = swAnn.GetTextFormat(i)

        'Change text to be 10mm, bold, italic, and Comic Sans MS font
        swTextFormat.CharHeight = 0.01
        swTextFormat.Bold = True
        swTextFormat.Italic = True
        swTextFormat.TypeFaceName = "Comic Sans MS"
        bRet = swAnn.SetTextFormat(i, False, swTextFormat)
    Next

End Sub
```

Change Texture on Face in Specified Configuration Example (C#)

This example shows how to change the texture on the selected face in the specified configuration.

```
//-----
// Preconditions:
// 1. Ensure the specified textures exist.
// 2. Open a part or assembly with a Default configuration.
// 3. Select a face.
// 4. Open an Immediate window.
//
// Postconditions:
// 1. Applies texture to the selected face.
// 2. Inspect the Immediate window and the model.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ChangeTexture_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        ModelDocExtension swModelDocExt;
        Face2 face;
        Texture texture;
        bool boolstatus;
        string namStr;
        string configName;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = swModel.Extension;

            configName = "Default";

            face = (Face2)swSelMgr.GetSelectedObject6(1, -1);

            //Get existing texture on this face
            texture = face.GetTexture(configName);

            if ((texture != null))
            {
                Debug.Print("Texture before:");
                Debug.Print("Material: " + texture.MaterialName);
                Debug.Print("Granularity: " + texture.ScaleFactor);
                Debug.Print("Angle of rotation: " + texture.Angle);

                //Change texture on this face
                texture.Angle = 340;
                texture.ScaleFactor = 25;
                texture.MaterialName = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\data\\Images\\textures\\plastic\\brushed\\bt";
                boolstatus = face.SetTexture(configName, texture);

                Debug.Print("");
                Debug.Print("Texture after:");
                Debug.Print("Material: " + texture.MaterialName);
                Debug.Print("Granularity: " + texture.ScaleFactor);
                Debug.Print("Angle of rotation: " + texture.Angle);
            }
            else
            {
                //If no texture exists on this face, then apply a texture to it
                namStr = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\data\\Images\\textures\\pattern\\checker2.jpg";
                texture = swModelDocExt.CreateTexture(namStr, 5, 45, false);
                boolstatus = face.SetTexture(configName, texture);

                Debug.Print("New texture:");
                Debug.Print("Material: " + texture.MaterialName);
                Debug.Print("Granularity: " + texture.ScaleFactor);
                Debug.Print("Angle of rotation: " + texture.Angle);
            }

            Debug.Print(texture.GetSystemTextureName(texture.MaterialName.ToString(), out boolstatus));
        }
    }
}
```

```
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Change Texture on Face in Specified Configuration Example (VBA)

This example shows how to change the texture on the selected face in the specified configuration.

```

'-----
' Preconditions:
' 1. Ensure the specified textures exist.
' 2. Open a part or assembly with a Default configuration.
' 3. Select a face.
' 4. Open an Immediate window.
'
' Postconditions:
' 1. Applies texture to the selected face.
' 2. Inspect the Immediate window and the model.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim face As SldWorks.Face2
Dim texture As SldWorks.Texture
Dim boolstatus As Boolean
Dim namStr As String
Dim configName As String

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    configName = "Default"

    Set face = swSelMgr.GetSelectedObject6(1, -1)

    'Get existing texture on this face
    Set texture = face.GetTexture(configName)

    If Not texture Is Nothing Then

        Debug.Print "Texture before:"
        Debug.Print "Material: " & texture.MaterialName
        Debug.Print "Granularity: " & texture.ScaleFactor
        Debug.Print "Angle of rotation: " & texture.Angle

        'Change texture on this face
        texture.Angle = 340
        texture.ScaleFactor = 25
        texture.MaterialName = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\Images\textures\plastic\brushed\bblue.jpg"
        boolstatus = face.SetTexture(configName, texture)

        Debug.Print ""
        Debug.Print "Texture after:"
        Debug.Print "Material: " & texture.MaterialName
        Debug.Print "Granularity: " & texture.ScaleFactor
        Debug.Print "Angle of rotation: " & texture.Angle

    Else

        'If no texture exists on this face, then apply a texture to it
        namStr = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\Images\textures\pattern\checker2.jpg"
        Set texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
        boolstatus = face.SetTexture(configName, texture)

        Debug.Print "New texture:"
        Debug.Print "Material: " & texture.MaterialName
        Debug.Print "Granularity: " & texture.ScaleFactor
        Debug.Print "Angle of rotation: " & texture.Angle

    End If

    Debug.Print texture.GetSystemTextureName(texture.MaterialName, boolstatus)

End Sub

```

Change Texture on Face in Specified Configuration Example (VB.NET)

This example shows how to change the texture on the selected face in the specified configuration.

```

'-----
' Preconditions:
' 1. Ensure the specified textures exist.
' 2. Open a part or assembly with a Default configuration.
' 3. Select a face.
' 4. Open an Immediate window.
'
' Postconditions:
' 1. Applies texture to the selected face.
' 2. Inspect the Immediate window and the model.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swModelDocExt As ModelDocExtension
    Dim face As Face2
    Dim texture As Texture
    Dim boolstatus As Boolean
    Dim namStr As String
    Dim configName As String

```

Sub main()

```

    swModel = swApp.ActiveDoc
    swSelMgr = swModel.SelectionManager
    swModelDocExt = swModel.Extension

```

```

    configName = "Default"

```

```

    face = swSelMgr.GetSelectedObject6(1, -1)

```

```

    'Get existing texture on this face
    texture = face.GetTexture(configName)

```

If Not texture Is Nothing Then

```

    Debug.Print("Texture before:")
    Debug.Print("Material: " & texture.MaterialName)
    Debug.Print("Granularity: " & texture.ScaleFactor)
    Debug.Print("Angle of rotation: " & texture.Angle)

```

```

    'Change texture on this face

```

```

    texture.Angle = 340
    texture.ScaleFactor = 25
    texture.MaterialName = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\Images\textures\plastic\brushed\bblue.jpg"
    boolstatus = face.SetTexture(configName, texture)

```

```

    Debug.Print("")
    Debug.Print("Texture after:")
    Debug.Print("Material: " & texture.MaterialName)
    Debug.Print("Granularity: " & texture.ScaleFactor)
    Debug.Print("Angle of rotation: " & texture.Angle)

```

Else

```

    'If no texture exists on this face, then apply a texture to it
    namStr = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\Images\textures\pattern\checker2.jpg"
    texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
    boolstatus = face.SetTexture(configName, texture)

```

```

    Debug.Print("New texture:")
    Debug.Print("Material: " & texture.MaterialName)
    Debug.Print("Granularity: " & texture.ScaleFactor)
    Debug.Print("Angle of rotation: " & texture.Angle)

```

End If

```

    Debug.Print(texture.GetSystemTextureName(texture.MaterialName, boolstatus))

```

End Sub

Public swApp As SldWorks

End Class

Change to Isometric and Zoom to Fit View Mode Example (VBA)

This example shows how to change the current view mode to isometric and Zoom to Fit.

```
'-----  
'  
' Preconditions: Model document is open.  
'  
' Postconditions: Current view mode is changed to isometric and Zoom to Fit.  
'  
'-----  
Option Explicit  
Sub main()  
    Dim swApp          As SldWorks.SldWorks  
    Dim swModel         As SldWorks.ModelDoc2  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    ' Not quite the same as when done through the user interface;  
    ' model is zoomed out a bit further  
    swModel.ShowNamedView2 "Isometric", -1  
  
    ' Now view the same as done through the user interface  
    swModel.ViewZoomtofit2  
End Sub  
'-----
```

Change Visibility of Sketch Block Instances (C#)

This example shows how to hide and show sketch block instances in a drawing document.

```
//-----
// Preconditions:
// 1. Drawing document containing a sketch block
//    with one or more sketch block instances is open.
// 2. The sketch block is selected in the FeatureManager
//    design tree.
//
// Postconditions: All sketch block instances are hidden if visible, or
// shown if hidden.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace VisibleSketchBlockInstanceCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc;
            SelectionMgr swSelMgr;
            Feature swFeature;
            SketchBlockDefinition swBlockDefinition;
            SketchBlockInstance swBlockInstance;
            object[] blocks = null;
            int i = 0;

            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;

            // Block feature is selected in FeatureManager design tree
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            if (swFeature == null)
            {
                MessageBox.Show("Select a sketch block in the FeatureManager design tree, then rerun the macro.");
            }
            else
            {
                swBlockDefinition = (SketchBlockDefinition)swFeature.GetSpecificFeature2();
                Debug.Print("Feature type : " + swFeature.GetTypeName2());
                if ((swBlockDefinition != null))
                {
                    blocks = (object[])swBlockDefinition.GetInstances();
                    for (i = blocks.GetLowerBound(0); i <= blocks.GetUpperBound(0); i++)
                    {
                        swBlockInstance = (SketchBlockInstance)blocks[i];
                        Debug.Print("Sketch block instance: " + (i + 1));
                        Debug.Print(" Angle : " + swBlockInstance.Angle);
                        Debug.Print(" Scale : " + swBlockInstance.Scale2);

                        // Hide or show the sketch block instance
                        long status = 0;
                        status = swBlockInstance.Visible;
                        switch (status)
                        {
                            case (int)swAnnotationVisibilityState_e.swAnnotationHidden:
                                swBlockInstance.Visible = (int)swAnnotationVisibilityState_e.swAnnotationVisible;
                                Debug.Print(" Was hidden, now visible.");
                                break;
                            case (int)swAnnotationVisibilityState_e.swAnnotationVisible:
                                swBlockInstance.Visible = (int)swAnnotationVisibilityState_e.swAnnotationHidden;
                                Debug.Print(" Was visible, now hidden.");
                                break;
                            case (int)swAnnotationVisibilityState_e.swAnnotationHalfHidden:
                                MessageBox.Show("This block is half hidden.");
                                break;
                            case (int)swAnnotationVisibilityState_e.swAnnotationVisibilityUnknown:
                                MessageBox.Show("Failed to determine visibility of this block.");
                                break;
                        }
                    }
                }

                blocks = null;
            }
        }
    }
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;
```

```
}  
}
```

Change Visibility of Sketch Block Instances (VBA)

This example shows how to hide and show sketch block instances in a drawing document.

```
'-----
' Preconditions:
' 1. Drawing document containing a sketch
'    block with one or more sketch block instances is open.
' 2. The sketch block is selected in the FeatureManager design tree.
'
' Postconditions: All sketch block instances are hidden if visible, or
' shown if hidden.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swBlockDefinition As SldWorks.SketchBlockDefinition
Dim blocks As Variant
Dim i As Long

Sub main()

Set swApp = Application.SldWorks
Set swModelDoc = swApp.ActiveDoc
Set swSelMgr = swModelDoc.SelectionManager

' Select block is selected in FeatureManager design tree
Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
If swFeature Is Nothing Then
    MsgBox ("Select a sketch block in the FeatureManager design tree, then rerun the macro.")
Else
    Set swBlockDefinition = swFeature.GetSpecificFeature2
    Debug.Print "Feature type : " & swFeature.GetTypeName2
    If Not (swBlockDefinition Is Nothing) Then
        blocks = swBlockDefinition.GetInstances
        For i = LBound(blocks) To UBound(blocks)

            Dim swBlockInstance As SldWorks.SketchBlockInstance
            Set swBlockInstance = blocks(i)
            Debug.Print "Sketch block instance: " & (i + 1)
            Debug.Print " Angle : " & swBlockInstance.Angle
            Debug.Print " Scale : " & swBlockInstance.Scale2

            ' Hide or show the sketch block instance
            Dim status As Long
            status = swBlockInstance.Visible
            Select Case status
                Case swAnnotationHidden
                    swBlockInstance.Visible = swAnnotationVisible
                    Debug.Print " Was hidden, now visible."
                Case swAnnotationVisible
                    swBlockInstance.Visible = swAnnotationHidden
                    Debug.Print " Was visible, now hidden."
                Case swAnnotationHalfHidden
                    MsgBox ("This block is half hidden.")
                Case swAnnotationVisibilityUnknown
                    MsgBox ("Failed to determine visibility of this block.")
            End Select

        Next i
    End If

    blocks = Empty

End If

End Sub
```

Change Visibility of Sketch Block Instances (VB.NET)

This example shows how to hide and show sketch block instances in a drawing document.

```

'-----
' Preconditions:
' 1. Drawing document containing a sketch
'    block with one or more sketch block instances is open.
' 2. The sketch block is selected in the FeatureManager design tree.
'
' Postconditions: All sketch block instances are hidden if visible, or
' shown if hidden.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub Main()

        Dim swModelDoc As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swBlockDefinition As SketchBlockDefinition
        Dim blocks As Object
        Dim i As Integer

        swModelDoc = swApp.ActiveDoc
        swSelMgr = swModelDoc.SelectionManager

        ' Select block is selected in FeatureManager design tree
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        If swFeature Is Nothing Then
            MsgBox("Select a sketch block in the FeatureManager design tree, then rerun the macro.")
        Else
            swBlockDefinition = swFeature.GetSpecificFeature2
            Debug.Print("Feature type : " & swFeature.GetTypeName2)
            If Not (swBlockDefinition Is Nothing) Then
                blocks = swBlockDefinition.GetInstances
                For i = LBound(blocks) To UBound(blocks)

                    Dim swBlockInstance As SketchBlockInstance
                    swBlockInstance = blocks(i)
                    Debug.Print("Sketch block instance: " & (i + 1))
                    Debug.Print(" Angle : " & swBlockInstance.Angle)
                    Debug.Print(" Scale : " & swBlockInstance.Scale2)

                    ' Hide or show the sketch block instance
                    Dim status As Long
                    status = swBlockInstance.Visible
                    Select Case status
                        Case swAnnotationVisibilityState_e.swAnnotationHidden
                            swBlockInstance.Visible = swAnnotationVisibilityState_e.swAnnotationVisible
                            Debug.Print(" Was hidden, now visible.")
                        Case swAnnotationVisibilityState_e.swAnnotationVisible
                            swBlockInstance.Visible = swAnnotationVisibilityState_e.swAnnotationHidden
                            Debug.Print(" Was visible, now hidden.")
                        Case swAnnotationVisibilityState_e.swAnnotationHalfHidden
                            MsgBox("This block is half hidden.")
                        Case swAnnotationVisibilityState_e.swAnnotationVisibilityUnknown
                            MsgBox("Failed to determine visibility of this block.")
                    End Select

                    Next i
                End If

                blocks = Nothing
            End If
        End Sub

        ''' <summary>
        ''' The SldWorks swApp variable is pre-assigned for you.
        ''' </summary>
        Public swApp As SldWorks

    End Class

```

Change Weld Bead Faces Example (VBA)

This example shows how to change the faces of an existing fillet weld bead.

```
'-----  
'  
' Preconditions: Fillet weld bead exists and is selected in  
'             the open part document.  
'  
' Postconditions: Fillet weld bead is recreated on the  
'             the selected faces.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim SwFeat As SldWorks.Feature  
Dim swWeldBead As WeldmentBeadFeatureData  
Dim SwModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swComp As SldWorks.Component2  
Dim f1 As Variant  
Dim f2 As Variant  
Dim fa1(0) As Object  
Dim fa2(0) As Object  
Dim boolstatus As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set SwModel = swApp.ActiveDoc  
Set swSelMgr = SwModel.SelectionManager  
  
'Select the fillet weld bead feature  
Set SwFeat = swSelMgr.GetSelectedObject5(1)  
Set swWeldBead = SwFeat.GetDefinition  
  
'Roll back the model to the feature just above the weld bead feature  
boolstatus = swWeldBead.AccessSelections(SwModel, swComp)  
  
'Use both sides of the fillet weld bead  
swWeldBead.UseOtherSide = True  
  
'Set the type of weld bead to intermittent  
swWeldBead.BeadType(swWeldBead.OtherSide) = swWeldBead.TypeIntermittent  
  
'Select the two faces to which you want to apply the weld bead  
Set fa1(0) = swSelMgr.GetSelectedObject5(1)  
Set fa2(0) = swSelMgr.GetSelectedObject5(2)  
f1 = fa1  
f2 = fa2  
  
'Use the selected faces for the weld bead  
boolstatus = swWeldBead.SetFaces(swWeldBead.OtherSide, (f1), (f2))  
'Modify the weld bead using the just selected faces  
boolstatus = SwFeat.ModifyDefinition(swWeldBead, SwModel, swComp)  
  
End Sub
```

Change Width of FeatureManager Design Tree Example (VBA)

This example shows how to change the width of the FeatureManager design tree.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Doubles the width of FeatureManager design tree.  
' 2. Examine the Immediate window and FeatureManager  
'    design tree.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim nWidth As Long  
    Dim NewWidth As Long  
    Dim nRetVal As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    nWidth = swModel.GetFeatureManagerWidth  
    nRetVal = swModel.SetFeatureManagerWidth(nWidth * 2)  
    NewWidth = nWidth * 2  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Old width = " & nWidth & " pixels"  
    Debug.Print "  New width = " & NewWidth & " pixels"  
  
End Sub
```

Change Wrap Feature Face Example (C#)

This example shows how to insert a wrap feature and change the face on which to apply a wrap feature.

```
//-----
// Preconditions: Verify that the part to open exists.
//
// Postconditions:
// 1. Opens the part document.
// 2. Sketches a rectangle on the top plane.
// 3. Selects the sketch of the rectangle and the
// face where to scribe the sketch as a wrap feature.
// 4. Inserts the wrap feature.
// 5. Rotates the part about its center.
// 6. Selects another face on the part.
// 7. Edits the wrap feature and wraps the rectangular
// sketch on the second face.
//
// NOTE: Because the part document is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureManager = default(FeatureManager);
            ModelView swModelView = default(ModelView);
            Feature swWrapFeature = default(Feature);
            WrapSketchFeatureData swWrapFeatureData = default(WrapSketchFeatureData);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            PartDoc swPart = default(PartDoc);
            Face2 swFace = default(Face2);
            object[] sketchLines = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            // Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\clamp1.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Sketch a rectangle on the top plane
            swModel.ShowNamedView2("Top", 5);
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            swModel.ViewZoomtofit2();
            sketchLines = (object[])swSketchManager.CreateCornerRectangle(-0.00169758295694522, 0.000948506512727088, 0, 0.0013668226995581, -0.0009849875
32447629, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            // Select the sketch of the rectangle and the
            // face where to scribe the sketch as a wrap feature
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 3.03698880696047E-03, 0.036653462145523, -1.43855543627342E-03, true, 1, null, 0);

            // Insert the wrap feature
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.InsertWrapFeature(2, 0.001, false);
            swModel.ClearSelection2(true);

            // Rotate the part about its center to select another face
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.RotateAboutCenter(0.0113065031234594, 0);
            swModelView.RotateAboutCenter(0.00565325156172972, 0);
            swModelView.RotateAboutCenter(0.00565325156172972, 0);
            swModelView.RotateAboutCenter(0.0169597546851892, 0);
            swModelView.RotateAboutCenter(0.00565325156172972, 0);
            swModelView.RotateAboutCenter(0.0113065031234594, 0);
            swModelView.RotateAboutCenter(0.0169597546851892, 0);
            swModelView.RotateAboutCenter(0.00565325156172972, 0);
            swModelView.RotateAboutCenter(0.0113065031234594, 0);
            swModelView.RotateAboutCenter(0.0169597546851892, 0);
            swModelView.RotateAboutCenter(0.0282662578086486, 0);
            swModelView.RotateAboutCenter(0.0113065031234594, 0);
            swModelView.RotateAboutCenter(0.0226130062469189, 0);
            swModelView.RotateAboutCenter(0.0282662578086486, 0);
            swModelView.RotateAboutCenter(0.0113065031234594, 0);
            swModelView.RotateAboutCenter(0.0282662578086486, 0);
            swModelView.RotateAboutCenter(0.0169597546851892, 0);
            swModelView.RotateAboutCenter(0.0339195093703783, 0);
            swModelView.RotateAboutCenter(0.039572760932108, 0);
            swModelView.RotateAboutCenter(0.0282662578086486, 0);
            swModelView.RotateAboutCenter(0.0226130062469189, 0);
            swModelView.RotateAboutCenter(0.0339195093703783, 0);
            swModelView.RotateAboutCenter(0.0508792640555675, 0);
            swModelView.RotateAboutCenter(0.0226130062469189, -0.00845813406735716);
            swModelView.RotateAboutCenter(0.0339195093703783, 0);
            swModelView.RotateAboutCenter(0.0508792640555675, -0.00845813406735716);
            swModelView.RotateAboutCenter(0.0226130062469189, 0);
            swModelView.RotateAboutCenter(0.039572760932108, -0.00845813406735716);
        }
    }
}
```

[illegible]

Change Wrap Feature Face Example (VBA)

This example shows how to insert a wrap feature and change the face on which to apply a wrap feature.

```

'-----
' Preconditions: Verify that the part to open exists.
'
' Postconditions:
' 1. Opens the part document.
' 2. Sketches a rectangle on the top plane.
' 3. Selects the sketch of the rectangle and the
'    face where to scribe the sketch as a wrap feature.
' 4. Inserts the wrap feature.
' 5. Rotates the part about its center.
' 6. Selects another face on the part.
' 7. Edits the wrap feature and wraps the rectangular
'    sketch on the second face.
'
' NOTE: Because the part document is used elsewhere, do not save changes.
'-----
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swFeature As SldWorks.Feature
Dim swFeatureManager As SldWorks.FeatureManager
Dim swModelView As SldWorks.ModelView
Dim swWrapFeature As SldWorks.Feature
Dim swWrapFeatureData As SldWorks.WrapSketchFeatureData
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swFace As SldWorks.Face2
Dim sketchLines As Variant
Dim status As Boolean
Dim errors As Long, warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks

    ' Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    ' Sketch a rectangle on the top plane
    swModel.ShowNamedView2 "Top", 5
    status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    swModel.ViewZoomtofit2
    sketchLines = swSketchManager.CreateCornerRectangle(-1.69758295694522E-03, 9.48506512727088E-04, 0, 1.3668226995581E-03, -9.84987532447629E-04, 0)
    swModel.ClearSelection2 True

    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    ' Select the sketch of the rectangle and the
    ' face where to scribe the sketch as a wrap feature
    status = swModelDocExt.SelectByID2("Sketch11", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", 3.03698880696047E-03, 0.036653462145523, -1.43855543627342E-03, True, 1, Nothing, 0)

    ' Insert the wrap feature
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.InsertWrapFeature(2, 0.001, False)
    swModel.ClearSelection2 True

    ' Rotate the part about its center to select another face
    Set swModelView = swModel.ActiveView
    swModelView.RotateAboutCenter 1.13065031234594E-02, 0
    swModelView.RotateAboutCenter 5.65325156172972E-03, 0
    swModelView.RotateAboutCenter 5.65325156172972E-03, 0
    swModelView.RotateAboutCenter 1.69597546851892E-02, 0
    swModelView.RotateAboutCenter 5.65325156172972E-03, 0
    swModelView.RotateAboutCenter 1.13065031234594E-02, 0
    swModelView.RotateAboutCenter 1.69597546851892E-02, 0
    swModelView.RotateAboutCenter 5.65325156172972E-03, 0
    swModelView.RotateAboutCenter 1.13065031234594E-02, 0
    swModelView.RotateAboutCenter 1.69597546851892E-02, 0
    swModelView.RotateAboutCenter 2.82662578086486E-02, 0
    swModelView.RotateAboutCenter 1.13065031234594E-02, 0
    swModelView.RotateAboutCenter 2.26130062469189E-02, 0
    swModelView.RotateAboutCenter 2.82662578086486E-02, 0
    swModelView.RotateAboutCenter 1.13065031234594E-02, 0
    swModelView.RotateAboutCenter 2.82662578086486E-02, 0
    swModelView.RotateAboutCenter 1.69597546851892E-02, 0
    swModelView.RotateAboutCenter 3.39195093703783E-02, 0
    swModelView.RotateAboutCenter 0.039572760932108, 0
    swModelView.RotateAboutCenter 2.82662578086486E-02, 0
    swModelView.RotateAboutCenter 2.26130062469189E-02, 0
    swModelView.RotateAboutCenter 3.39195093703783E-02, 0
    swModelView.RotateAboutCenter 5.08792640555675E-02, 0
    swModelView.RotateAboutCenter 2.26130062469189E-02, -8.45813406735716E-03
    swModelView.RotateAboutCenter 3.39195093703783E-02, 0

```

[illegible]


```

Preconditions: Verify that the part document to open exists.

Postconditions:
1. Opens the part document.
2. Sketches a rectangle on the top plane.
3. Selects the sketch of the rectangle and the
   face where to scribe the sketch as a wrap feature.
4. Inserts the wrap feature.
5. Rotates the part about its center.
6. Selects another face on the part.
7. Edits the wrap feature and wraps the rectangular
   sketch on the second face.

NOTE: Because the part document is used elsewhere, do not save changes.

```

Partial Class SolidWorksMacro

[illegible]

[illegible]

Check Bodies for Faults Example (VBA)

This example shows how to check the solid and sheet bodies in a part for faults.

```
'-----
' Preconditions:
' 1. Open a part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Checks the bodies in the part document.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub ProcessFaultEntity(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swFaultEnt As SldWorks.FaultEntity)
    Dim nCount As Long
    Dim swEnt As SldWorks.Entity
    Dim bRet As Boolean
    Dim i As Long

    nCount = swFaultEnt.Count: If 0 = nCount Then Exit Sub 'Else print the error code for each fault
    For i = 0 To nCount - 1
        Set swEnt = swFaultEnt.Entity(i)
        If Not swEnt Is Nothing Then
            bRet = swEnt.Select4(True, Nothing): Debug.Assert bRet
        End If
        Debug.Print "    Fault[" & i & "] = " & swFaultEnt.ErrorCode(i)
    Next i
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBodyArr As Variant
    Dim vBody As Variant
    Dim swBody As SldWorks.Body2
    Dim nRetVal1 As Long
    Dim nRetVal2 As Long
    Dim swFaultEnt As SldWorks.FaultEntity
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel

    Debug.Print "File = " & swModel.GetPathName

    vBodyArr = swPart.GetBodies2(swAllBodies, True): Debug.Assert Not IsEmpty(vBodyArr)
    For Each vBody In vBodyArr
        Set swBody = vBody
        Set swFaultEnt = swBody.Check3
        Debug.Print "    Body[" & swBody.GetType & "] --> " & swBody.GetSelectionId
        nRetVal1 = swBody.Check 'Obsolete method
        nRetVal2 = swBody.Check2 'Obsolete method
        Debug.Print "        IBody2::Check (1 if valid; 0 if not) = " & nRetVal1
        Debug.Print "        IBody2::Check2(Number of faults) = " & nRetVal2
        ProcessFaultEntity swApp, swModel, swFaultEnt
    Next vBody
End Sub
```

Check Edges for Faults Example (C#)

This example shows how to check the edges in a part for faults.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Checks the bodies and edges in the part.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetEdgesCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            object[] vBodyArr = null;
            object vBody = null;
            Body2 swBody = default(Body2);
            int nRetVal1 = 0;
            int nRetVal2 = 0;
            object[] vEdgeArr = null;
            object vEdge = null;
            Edge swEdge = default(Edge);
            FaultEntity swFaultEnt = default(FaultEntity);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swPart = (PartDoc)swModel;
            Debug.Print("File = " + swModel.GetPathName());
            vBodyArr = (object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, true);
            Debug.Assert((vBodyArr != null));
            foreach (object vBody_loopVariable in vBodyArr)
            {
                vBody = vBody_loopVariable;
                swBody = (Body2)vBody;
                Debug.Print(" Body[" + swBody.GetType() + "] --> " + swBody.GetSelectionId());
                nRetVal1 = swBody.Check(); // Obsolete method
                nRetVal2 = swBody.Check2(); // Obsolete method
                Debug.Print(" IBody2::Check (1 if valid; 0 if not) = " + nRetVal1);
                Debug.Print(" IBody2::Check2(Number of faults) = " + nRetVal2);
                vEdgeArr = (object[])swBody.GetEdges();
                if ((vEdgeArr != null))
                {
                    foreach (object vEdge_loopVariable in vEdgeArr)
                    {
                        vEdge = vEdge_loopVariable;
                        swEdge = (Edge)vEdge;
                        swFaultEnt = swEdge.Check;
                        ProcessFaultEntity(swApp, swModel, swEdge, swFaultEnt);
                    }
                }
                else
                {
                    Debug.Print(" No edges in part.");
                }
            }
        }

        public string GetStringFromID(SldWorks swApp, ModelDoc2 swModel, byte[] vPIDarr)
        {
            string functionReturnValue = null;
            byte vPID;
            functionReturnValue = "";
            foreach (object vPID_loopVariable in vPIDarr)
            {
                vPID = (byte)vPID_loopVariable;
                functionReturnValue = functionReturnValue + vPID.ToString("###000");
            }
            return functionReturnValue;
        }

        public void ProcessFaultEntity(SldWorks swApp, ModelDoc2 swModel, Edge swEdge, FaultEntity swFaultEnt)
        {
            ModelDocExtension swModExt = default(ModelDocExtension);
            byte[] vPIDarr = null;
            int nCount = 0;
            Entity swEnt = default(Entity);
            bool bRet = false;
            int i = 0;

            nCount = swFaultEnt.Count;
            // For each edge, print:
            // * no fault found, if no fault found
            // * edge ID and error code, if fault found
            if (0 == nCount)
            {
                Debug.Print(" No fault in edge.");
                return;
            }

            swModExt = (ModelDocExtension)swModel.Extension;
            vPIDarr = (byte[])swModExt.GetPersistReference3(swEdge);
            Debug.Assert((vPIDarr != null));
            Debug.Print(" Edge ID = " + GetStringFromID(swApp, swModel, vPIDarr));
            for (i = 0; i <= nCount - 1; i++)
            {
                swEnt = (Entity)swFaultEnt.get_Entity(i);
                if ((swEnt != null))
            }
        }
    }
}
```

```

        {
            bRet = swEnt.Select4(true, null);
            Debug.Assert(bRet);
        }
        Debug.Print("        Fault[" + i + "]                = " + swFaultEnt.get_ErrorCode(i));
    }
}

/// <summary>
/// The Sldworks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Check Edges for Faults Example (VBA)

This example shows how to check the edges in a part for faults.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Checks the bodies and edges in the part.
' 2. Examine the Immediate window.
'-----
Option Explicit

Function GetStringFromID(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vPIDarr As Variant) As String
    Dim vPID As Variant

    For Each vPID In vPIDarr
        GetStringFromID = GetStringFromID & Format(vPID, "###000")
    Next vPID
End Function

Sub ProcessFaultEntity(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swEdge As SldWorks.Edge, swFaultEnt As SldWorks.FaultEntity)
    Dim swModExt As SldWorks.ModelDocExtension
    Dim vPIDarr As Variant
    Dim nCount As Long
    Dim swEnt As SldWorks.Entity
    Dim bRet As Boolean
    Dim i As Long

    nCount = swFaultEnt.Count
    ' For each edge, print:
    ' * no fault found, if not fault found
    ' * edge ID and error code, if fault found
    If 0 = nCount Then
        Debug.Print "    No fault found."
        Exit Sub
    End If
    Set swModExt = swModel.Extension
    vPIDarr = swModExt.GetPersistReference3(swEdge): Debug.Assert Not IsEmpty(vPIDarr)
    Debug.Print "    Edge ID = " & GetStringFromID(swApp, swModel, vPIDarr)

    For i = 0 To nCount - 1
        Set swEnt = swFaultEnt.Entity(i)
        If Not swEnt Is Nothing Then
            bRet = swEnt.Select4(True, Nothing): Debug.Assert bRet
        End If
        Debug.Print "        Fault[" & i & "]                = " & swFaultEnt.ErrorCode(i)
    Next i
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBodyArr As Variant
    Dim vBody As Variant
    Dim swBody As SldWorks.Body2
    Dim nRetVal1 As Long
    Dim nRetVal2 As Long
    Dim vEdgeArr As Variant
    Dim vEdge As Variant
    Dim swEdge As SldWorks.Edge
    Dim swFaultEnt As SldWorks.FaultEntity

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel

    Debug.Print "File = " & swModel.GetPathName

    vBodyArr = swPart.GetBodies2(swAllBodies, True): Debug.Assert Not IsEmpty(vBodyArr)
    For Each vBody In vBodyArr
        Set swBody = vBody
        Debug.Print "    Body[" & swBody.GetType & "] --> " & swBody.GetSelectionId
        nRetVal1 = swBody.Check ' Obsolete method
        nRetVal2 = swBody.Check2 ' Obsolete method
        Debug.Print "        IBody2::Check (1 if valid; 0 if not) = " & nRetVal1
        Debug.Print "        IBody2::Check2 (Number of faults) = " & nRetVal2

        vEdgeArr = swBody.GetEdges
        If Not IsEmpty(vEdgeArr) Then
            For Each vEdge In vEdgeArr
                Set swEdge = vEdge
                Set swFaultEnt = swEdge.Check
                ProcessFaultEntity swApp, swModel, swEdge, swFaultEnt
            Next vEdge
        Else
            Debug.Print "        No edges in part."
        End If
    Next vBody
End Sub

```

Check Edges for Faults Example (VB.NET)

This example shows how to check the edges in a part for faults.

```

-----
Preconditions:
1. Open a part document.
2. Open the Immediate window.
-----
Postconditions:
1. Checks the bodies and edges in the part.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim vBodyArr As Object
        Dim vBody As Object
        Dim swBody As Body2
        Dim nRetVal1 As Integer
        Dim nRetVal2 As Integer
        Dim vEdgeArr As Object
        Dim vEdge As Object
        Dim swEdge As Edge
        Dim swFaultEnt As FaultEntity

        swModel = swApp.ActiveDoc
        swPart = swModel
        Debug.Print("File = " & swModel.GetPathName)
        vBodyArr = swPart.GetBodies2(swBodyType_e.swAllBodies, True) : Debug.Assert(Nothing(vBodyArr))
        For Each vBody In vBodyArr
            swBody = vBody
            Debug.Print(" Body[" & swBody.GetType & "] --> " & swBody.GetSelectionId)
            nRetVal1 = swBody.Check ' Obsolete method
            nRetVal2 = swBody.Check2 ' Obsolete method
            Debug.Print(" IBody2::Check (1 if valid; 0 if not) = " & nRetVal1)
            Debug.Print(" IBody2::Check2(Number of faults) = " & nRetVal2)
            vEdgeArr = swBody.GetEdges
            If Not IsNothing(vEdgeArr) Then
                For Each vEdge In vEdgeArr
                    swEdge = vEdge
                    swFaultEnt = swEdge.Check
                    ProcessFaultEntity(swApp, swModel, swEdge, swFaultEnt)
                Next vEdge
            Else
                Debug.Print(" No edges in part.")
            End If
        Next vBody
    End Sub

    Function GetStringFromID(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal vPIDarr As Object) As String
        Dim vPID As Object
        GetStringFromID = ""
        For Each vPID In vPIDarr
            GetStringFromID = GetStringFromID & Format(vPID, "###000")
        Next vPID
    End Function

    Sub ProcessFaultEntity(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swEdge As Edge, ByVal swFaultEnt As FaultEntity)
        Dim swModExt As ModelDocExtension
        Dim vPIDarr As Object
        Dim nCount As Integer
        Dim swEnt As Entity
        Dim bRet As Boolean
        Dim i As Integer

        nCount = swFaultEnt.Count
        ' For each edge, print:
        ' * no fault found, if no fault found
        ' * edge ID and error code, if fault found
        If 0 = nCount Then
            Debug.Print(" No fault in edge.")
            Exit Sub
        End If

        swModExt = swModel.Extension
        vPIDarr = swModExt.GetPersistReference3(swEdge) : Debug.Assert(Nothing(vPIDarr))
        Debug.Print(" Edge ID = " & GetStringFromID(swApp, swModel, vPIDarr))
        For i = 0 To nCount - 1
            swEnt = swFaultEnt.Entity(i)
            If Not swEnt Is Nothing Then
                bRet = swEnt.Select4(True, Nothing) : Debug.Assert(bRet)
            End If
            Debug.Print(" Fault[" & i & "] = " & swFaultEnt.ErrorCode(i))
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Check Faces for Faults Example (VBA)

This example shows how to check faces for faults.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Checks the faces in the part for faults.
' 2. Examine the Immediate window.
'-----
Option Explicit

Function GetStringFromID(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vPIDarr As Variant) As String
    Dim vPID As Variant
    For Each vPID In vPIDarr
        Debug.Assert vbByte = VarType(vPID)
        GetStringFromID = GetStringFromID & Format(vPID, "###000")
    Next vPID
End Function

Sub ProcessFaultEntity(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swFace As SldWorks.Face2, swFaultEnt As SldWorks.FaultEntity) Dim
swModExt As SldWorks.ModelDocExtension
    Dim vPIDarr As Variant
    Dim nCount As Long
    Dim swEnt As SldWorks.Entity
    Dim bRet As Boolean
    Dim i As Long
    nCount = swFaultEnt.Count
    ' For each face, print:
    ' * no fault found, if not fault found
    ' * face ID and error code, if fault found
    If 0 = nCount Then
        Debug.Print "    No fault found."
        Exit Sub
    End If
    Set swModExt = swModel.Extension
    vPIDarr = swModExt.GetPersistReference3(swFace): Debug.Assert Not IsEmpty(vPIDarr)
    Debug.Print "    Face ID = " & GetStringFromID(swApp, swModel, vPIDarr)
    For i = 0 To nCount - 1
        Set swEnt = swFaultEnt.Entity(i)
        If Not swEnt Is Nothing Then
            bRet = swEnt.Select4(True, Nothing): Debug.Assert bRet
        End If
        Debug.Print "    Fault[" & i & "] = " & swFaultEnt.ErrorCode(i)
    Next i
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBodyArr As Variant
    Dim vBody As Variant
    Dim swBody As SldWorks.Body2
    Dim nRetVal1 As Long
    Dim nRetVal2 As Long
    Dim swFace As SldWorks.Face2
    Dim swFaultEnt As SldWorks.FaultEntity

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel

    Debug.Print "File = " & swModel.GetPathName

    vBodyArr = swPart.GetBodies2(swAllBodies, True): Debug.Assert Not IsEmpty(vBodyArr)
    For Each vBody In vBodyArr
        Set swBody = vBody
        Debug.Print "    Body[" & swBody.GetType & "] --> " & swBody.GetSelectionId
        nRetVal1 = swBody.Check ' Obsolete method
        nRetVal2 = swBody.Check2 ' Obsolete method
        Debug.Print "    IBody2::Check (1 if valid; 0 if not)= " & nRetVal1
        Debug.Print "    IBody2::Check2 (Number of faults) = " & nRetVal2
        Set swFace = swBody.GetFirstFace
        Do While Not swFace Is Nothing
            Set swFaultEnt = swFace.Check
            ProcessFaultEntity swApp, swModel, swFace, swFaultEnt
            Set swFace = swFace.GetNextFace
        Loop
    Next vBody
End Sub

```

Check Interference Among Bodies Example (VBA)

This example shows how to check interference among bodies in a part document.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Finds a body in a solid body folder that interferes with
'    one or more solid bodies in another solid body folder.
' 2. Examine the Immediate window and expand Cut list(31) >
'    Sub-weldment(8).
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc
Dim swPartDoc As SldWorks.ModelDoc2
Dim swFeature As SldWorks.Feature
Dim swBodyFolder As SldWorks.BodyFolder
Dim BodyFolder As SldWorks.BodyFolder
Dim toolBodyList As Variant
Dim targetBodyList As Variant
Dim contLoop As Boolean
Dim areBodyClash As Boolean
Dim swModeler As SldWorks.Modeler
Dim vBodies As Variant
Dim vFaces1 As Variant
Dim vFaces2 As Variant
Dim sldBody1 As SldWorks.Body2
Dim sldBody2 As SldWorks.Body2
Dim sizeOfOut1, sizeOfOut2, sizeOfOut3 As Long

Sub GetVariantOfBody(swFeature As SldWorks.Feature, bodyList As Variant)
    Dim tt As Variant
    Dim FeatType As String
    Dim FeatTypeName As String

    FeatType = swFeature.Name
    FeatTypeName = swFeature.GetTypeName2
    Debug.Print FeatType & " [" & FeatTypeName & "]"

    Set swBodyFolder = swFeature.GetSpecificFeature2
    Dim count As Long
    count = swBodyFolder.GetBodyCount
    If (count < 1) Then
        MsgBox ("There are no bodies. Create a body in the folder.")
    Else
        bodyList = swBodyFolder.GetBodies
    End If
End Sub

Sub main()

    Set swApp = Application.SldWorks
    Set swPartDoc = swApp.ActiveDoc
    Set swFeature = swPartDoc.FirstFeature
    Set swModeler = swApp.GetModeler

    contLoop = True

    While Not swFeature Is Nothing And contLoop = True
        Dim Name As String
        Name = swFeature.GetTypeName2
        If (Name = "SolidBodyFolder") Then
            contLoop = False

            GetVariantOfBody swFeature, targetBodyList

            Set swFeature = swFeature.GetFirstSubFeature

            GetVariantOfBody swFeature, toolBodyList

            Dim i As Long

            areBodyClash = swModeler.CheckInterference3(targetBodyList, toolBodyList, (swBodyInterference_ReturnInterferingObject +
            swBodyInterference_IncludeCoincidentFaces), vFaces1, vFaces2, vBodies)
            If (areBodyClash) Then
                sizeOfOut1 = UBound(vFaces1)
                sizeOfOut2 = UBound(vFaces2)
                sizeOfOut3 = UBound(vBodies)
                Debug.Print (" Returning interfering body (faces touching each other):")
                Debug.Print (" No. of faces that interfered from the first body with the second body")
                Debug.Print (" " & sizeOfOut1 + 1)
                Debug.Print (" No. of faces that interfered from the second body with the first body:")
                Debug.Print (" " & sizeOfOut2 + 1)
                Debug.Print (" No. of interfering bodies")
                Debug.Print (" " & sizeOfOut3 + 1)
            End If
        End If
    End While
End Sub

```

```

End If

Debug.Print " "

areBodyClash = swModeler.CheckInterference3(targetBodyList, toolBodyList, (swBodyInterference_OptionDefault), vFaces1, vFaces2, vBodies)
If (areBodyClash) Then
    sizeOfOut1 = UBound(vFaces1)
    sizeOfOut2 = UBound(vFaces2)
    sizeOfOut3 = UBound(vBodies)
    Debug.Print ("    Returning interfering body (faces not touching each other): ")
    Debug.Print ("        No. of faces1")
    Debug.Print ("        " & sizeOfOut1 + 1)
    Debug.Print ("        No. of faces2")
    Debug.Print ("        " & sizeOfOut2 + 1)
    Debug.Print ("        No. of bodies")
    Debug.Print ("        " & sizeOfOut3 + 1)
End If
Else
    Set swFeature = swFeature.GetNextFeature
End If
Wend

End Sub

```

Check for Interference Between Two Bodies Example (VBA)

This example shows how to check for interference between two bodies in an assembly.

```

'-----
' Preconditions:
' Open public_documents\samples\tutorial\api\Cube_Assem.sldasm.
'
' Postconditions:
' 1. Gets the bodies of two components and transforms them in the
'    context of the assembly.
' 2. Checks for interference between the two bodies.
' 3. Sketches lines around the faces involved in the interference.
' 4. Inspect the graphics area after selecting 3DSketch1 in the
'    FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Sub main()
```

```

    Dim swModel As SldWorks.ModelDoc2
    Dim swComponent1 As SldWorks.Component2
    Dim swComponent2 As SldWorks.Component2
    Dim bValue As Boolean
    Dim swBody1 As SldWorks.Body2
    Dim swBody2 As SldWorks.Body2
    Dim swBodyCopy1 As SldWorks.Body2
    Dim swBodyCopy2 As SldWorks.Body2
    Dim vBodies As Variant
    Dim swModeler As SldWorks.Modeler
    Dim vFaces1 As Variant
    Dim vFaces2 As Variant
    Dim vFace As Variant
    Dim swFace As SldWorks.Face2
    Dim vEdges As Variant
    Dim vEdge As Variant
    Dim swEdge As SldWorks.Edge
    Dim swVertex1 As SldWorks.Vertex
    Dim swVertex2 As SldWorks.Vertex
    Dim vPoint1 As Variant
    Dim vPoint2 As Variant
    Dim swTransform1 As SldWorks.MathTransform
    Dim swTransform2 As SldWorks.MathTransform
    Dim lIdx As Long
    Dim lNumEdges As Long
    Dim lCoord As Long
    Dim lEpsilon As Double

    lEpsilon = 0.000000001

    Set swApp = Application.SldWorks
    Set swModeler = swApp.GetModeler
    Set swModel = swApp.ActiveDoc

    swModel.ClearSelection2 True
    bValue = swModel.Extension.SelectByID("Cube-2@Cube_Assem", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComponent1 = swModel.SelectionManager.GetSelectedObject6(1, -1)

    bValue = swModel.Extension.SelectByID("Cube-3@Cube_Assem", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComponent2 = swModel.SelectionManager.GetSelectedObject6(1, -1)

    'Get transform of each component
    Set swTransform1 = swComponent1.Transform
    Set swTransform2 = swComponent2.Transform

    'Get bodies of first component
    vBodies = swComponent1.GetBodies2(swBodyType_e.swSolidBody)

    Set swBody1 = vBodies(0)
    Set swBodyCopy1 = swBody1.Copy

    'Get bodies of second component
    vBodies = swComponent2.GetBodies2(swBodyType_e.swSolidBody)

    Set swBody2 = vBodies(0)
    Set swBodyCopy2 = swBody2.Copy

    'Apply transform to move copies to their correct positions in the assembly
    bValue = swBodyCopy1.ApplyTransform(swTransform1)
    bValue = swBodyCopy2.ApplyTransform(swTransform2)

    swModel.EditRebuild3

    bValue = swModeler.CheckInterferenceBetweenTwoBodies(swBodyCopy1, swBodyCopy2, False, vFaces1, vFaces2, vBodies)

    'Display edges of faces interfering in first body
    If (Not IsEmpty(vFaces1)) Then
        swModel.SetAddToDB True
        swModel.Insert3DSketch2 False
    End If

```

```

For Each vFace In vFaces1
    Set swFace = vFace
    vEdges = swFace.GetEdges
    lNumEdges = (UBound(vEdges) - LBound(vEdges) + 1)
    For lIdx = 0 To (lNumEdges - 1)
        Set swEdge = vEdges(lIdx)
        Set swVertex1 = swEdge.GetStartVertex
        Set swVertex2 = swEdge.GetEndVertex
        vPoint1 = swVertex1.GetPoint
        vPoint2 = swVertex2.GetPoint
        For lCoord = 0 To 2
            If (Abs(vPoint1(lCoord)) < lEpsilon) Then
                vPoint1(lCoord) = 0#
            End If
            If (Abs(vPoint2(lCoord)) < lEpsilon) Then
                vPoint2(lCoord) = 0#
            End If
        Next lCoord
        swModel.CreateLine2 vPoint1(0), vPoint1(1), vPoint1(2), vPoint2(0), vPoint2(1), vPoint2(2)
    Next lIdx
Next vFace
swModel.Insert3DSketch2 True
swModel.SetAddToDB False
End If

'Display edges of faces interfering in second body
If (Not IsEmpty(vFaces2)) Then
    swModel.SetAddToDB True
    swModel.Insert3DSketch2 False
    For Each vFace In vFaces2
        Set swFace = vFace
        vEdges = swFace.GetEdges
        lNumEdges = (UBound(vEdges) - LBound(vEdges) + 1)
        For lIdx = 0 To (lNumEdges - 1)
            Set swEdge = vEdges(lIdx)
            Set swVertex1 = swEdge.GetStartVertex
            Set swVertex2 = swEdge.GetEndVertex
            vPoint1 = swVertex1.GetPoint
            vPoint2 = swVertex2.GetPoint
            For lCoord = 0 To 2
                If (Abs(vPoint1(lCoord)) < lEpsilon) Then
                    vPoint1(lCoord) = 0#
                End If
                If (Abs(vPoint2(lCoord)) < lEpsilon) Then
                    vPoint2(lCoord) = 0#
                End If
            Next lCoord
            swModel.CreateLine2 vPoint1(0), vPoint1(1), vPoint1(2), vPoint2(0), vPoint2(1), vPoint2(2)
        Next lIdx
    Next vFace
    swModel.Insert3DSketch2 True
    swModel.SetAddToDB False
End If
End Sub

```

Check Interference using IAssemblyDoc::ToolsCheckInterference2 Example (VBA)

This example shows how to check an assembly for interference using IAssemblyDoc::ToolsCheckInterference2.

```

'-----
' Preconditions:
' 1. Open a fully resolved assembly that contains at least
'    two interfering components.
' 2. Select at least two interfering components.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Selects the interfering faces.
' 2. Examine the graphics area and Immediate window, then
'    press F5.
' 3. Selects the interfering components.
' 4. Examine the graphics area and Immediate window, then
'    press F5.
'-----
Option Explicit

Sub main()

    Const bCoincidentInterference As Boolean = False

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim CompArray() As SldWorks.Component2
    Dim swSelData As SldWorks.SelectData
    Dim vCompArray As Variant
    Dim vIntCompArray As Variant
    Dim vIntFaceArray As Variant
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swComp As SldWorks.Component2
    Dim i As Long
    Dim nSelCount As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData

    Debug.Print "File = " & swModel.GetPathName

    nSelCount = swSelMgr.GetSelectedObjectCount
    ReDim CompArray(nSelCount - 1)
    Debug.Print "Number of selected components: " & nSelCount
    For i = 0 To (nSelCount - 1)
        Set CompArray(i) = swSelMgr.GetSelectedObjectsComponent2(i + 1)
        Debug.Print "  Comp[" & i & "] = " & CompArray(i).Name2
    Next i
    vCompArray = CompArray
    swAssy.ToolsCheckInterference2 nSelCount, (vCompArray), bCoincidentInterference, vIntCompArray, vIntFaceArray
    If (IsEmpty(vIntCompArray) = True) And (IsEmpty(vIntFaceArray) = True) Then
        Debug.Print "  No contact"
        Exit Sub
    End If

    If Not IsEmpty(vIntFaceArray) Then
        Debug.Print "    " & UBound(vIntFaceArray) + 1 & " faces interfere!"
        swModel.ClearSelection2 True
        For i = 0 To UBound(vIntFaceArray)
            Set swFace = vIntFaceArray(i)
            Set swEnt = swFace
            Set swComp = swEnt.GetComponent
            Debug.Print "      Component face[" & i & "] = " & swComp.Name2
            bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet
        Next i
        ' Interfering faces selected

        Stop
        ' Examine the graphics area and Immediate window, then
        ' press F5 to continue
    Else
        Debug.Assert Not IsEmpty(vIntCompArray)
        Debug.Assert False = bCoincidentInterference
        Debug.Print "  Faces touch but not checking for coincident interference!"
    End If

    If Not IsEmpty(vIntCompArray) Then
        Debug.Print "    " & UBound(vIntCompArray) + 1 & " Components interfere!"
        swModel.ClearSelection2 True
        For i = 0 To UBound(vIntCompArray)
            Set swComp = vIntCompArray(i)
            Debug.Print "      Component [" & i & "] = " & swComp.Name2
            bRet = swComp.Select2(True, 0): Debug.Assert bRet
        Next i
        ' Interfering components selected
    End If
End Sub

```

```
Stop  
' Examine the graphics area and Immediate window, then  
' press F5 to continue
```

```
End If
```

```
End Sub
```

Check Interference Using IModeler::CheckInterference Example (VBA)

This example shows how to check interference in an assembly using IModeler::CheckInterference.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\assem20.sldasm.
'
' Postconditions:
' 1. Checks interference between the components is
'    in the context of the assembly.
' 2. Draws lines that outline the faces in block20
'    that are involved in the interference with
'    cylinder20.
' 3. Creates a 3D sketch of the lines.
' 4. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

    Dim swModel As SldWorks.ModelDoc2
    Dim swPart1 As SldWorks.PartDoc
    Dim swPart2 As SldWorks.PartDoc
    Dim swComponent1 As SldWorks.Component2
    Dim swComponent2 As SldWorks.Component2
    Dim bValue As Boolean
    Dim swBody1 As SldWorks.Body2
    Dim swBody2 As SldWorks.Body2
    Dim swBodyCopy1 As SldWorks.Body2
    Dim swBodyCopy2 As SldWorks.Body2
    Dim vBodies As Variant
    Dim swModeler As SldWorks.Modeler
    Dim vFaces1 As Variant
    Dim vFaces2 As Variant
    Dim vFace As Variant
    Dim swFace As SldWorks.Face2
    Dim vEdges As Variant
    Dim vEdge As Variant
    Dim swEdge As SldWorks.Edge
    Dim swVertex1 As SldWorks.Vertex
    Dim swVertex2 As SldWorks.Vertex
    Dim vPoint1 As Variant
    Dim vPoint2 As Variant
    Dim swTransform1 As SldWorks.MathTransform
    Dim swTransform2 As SldWorks.MathTransform
    Dim swInverseTransform As SldWorks.MathTransform
    Dim swRelativeTransform As SldWorks.MathTransform
    Dim swMathPoint As SldWorks.MathPoint
    Dim swMathPointTransformed As SldWorks.MathPoint
    Dim swMathUtility As SldWorks.MathUtility
    Dim lIdx As Long
    Dim lNumEdges As Long
    Dim lCoord As Long
    Dim lEpsilon As Double

    lEpsilon = 0.000000001

    ' Connect to SOLIDWORKS Design
    Set swApp = Application.SldWorks

    ' Get modeler
    Set swModeler = swApp.GetModeler

    ' Get Math utility
    Set swMathUtility = swApp.GetMathUtility

    ' Get active document
    Set swModel = swApp.ActiveDoc

    ' Select first component
    swModel.ClearSelection2 True
    bValue = swModel.Extension.SelectByID("block20-1@assem20", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComponent1 = swModel.SelectionManager.GetSelectedObject6(1, -1)

    ' Select second component
    swModel.ClearSelection2 True
    bValue = swModel.Extension.SelectByID("cylinder20-1@assem20", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComponent2 = swModel.SelectionManager.GetSelectedObject6(1, -1)

    ' Get transform for each component
    Set swTransform1 = swComponent1.Transform
    Set swTransform2 = swComponent2.Transform

    ' Get document for each component
    Set swPart1 = swComponent1.GetModelDoc
    Set swPart2 = swComponent2.GetModelDoc

    ' Get bodies for first component
    vBodies = swComponent1.GetBodies2(swBodyType_e.swSolidBody)
    Set swBody1 = vBodies(0)

    ' Make a copy for boolean operations
    Set swBodyCopy1 = swBody1.Copy
```

```

' Get bodies for second component
vBodies = swComponent2.GetBodies2(swBodyType_e.swSolidBody)
Set swBody2 = vBodies(0)

' Make a copy for boolean operations
Set swBodyCopy2 = swBody2.Copy

' Move bodies to their correct position

' Apply transform
bValue = swBodyCopy1.ApplyTransform(swTransform1)
bValue = swBodyCopy2.ApplyTransform(swTransform2)

swModel.EditRebuild3

bValue = swModeler.CheckInterference(swBodyCopy1, swBodyCopy2, False, vFaces1, vFaces2, vBodies)

' Display edges of block20 that interfere with cylinder20
If (Not IsEmpty(vFaces1)) Then
    swModel.SetAddToDB True
    swModel.Insert3DSketch2 False
    For Each vFace In vFaces1
        Set swFace = vFace
        vEdges = swFace.GetEdges
        lNumEdges = (UBound(vEdges) - LBound(vEdges) + 1)
        For lIdx = 0 To (lNumEdges - 1)
            Set swEdge = vEdges(lIdx)
            Set swVertex1 = swEdge.GetStartVertex
            Set swVertex2 = swEdge.GetEndVertex
            vPoint1 = swVertex1.GetPoint
            vPoint2 = swVertex2.GetPoint
            For lCoord = 0 To 2
                If (Abs(vPoint1(lCoord)) < lEpsilon) Then
                    vPoint1(lCoord) = 0#
                End If
                If (Abs(vPoint2(lCoord)) < lEpsilon) Then
                    vPoint2(lCoord) = 0#
                End If
            Next lCoord
            swModel.CreateLine2 vPoint1(0), vPoint1(1), vPoint1(2), vPoint2(0), vPoint2(1), vPoint2(2)
        Next lIdx
    Next vFace
    swModel.Insert3DSketch2 True
    swModel.SetAddToDB False
End If

End Sub

```

Set and Toggle Selection Filters Example (VBA)

This example shows how to clear all existing selection filters, set new selection filters, and toggle the selection filters on and off.

```
'-----  
'  
' Preconditions: Model document is open.  
'  
' Postconditions:  
'   (1) Any existing selected selection filters are cleared.  
'   (2) The face, edge, and vertex selection filters are selected.  
'   (3) The selection filters are turned off.  
'  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim SelMgr As Object  
Dim boolstatus As Boolean
```

```
Sub main()
```

```
Dim params As Variant  
Dim selfils(2) As Long
```

```
Set swApp = Application.SldWorks
```

```
Set swModel = swApp.ActiveDoc  
Set SelMgr = swModel.SelectionManager
```

```
'Clear all existing selection filters  
params = swApp.GetSelectionFilters  
swApp.SetSelectionFilters params, False  
params = swApp.GetSelectionFilters
```

```
'Define filters to select  
selfils(0) = 1 'Edge  
selfils(1) = 2 'Face  
selfils(2) = 3 'Vertex  
swApp.SetSelectionFilters (selfils), True
```

```
'Toggle the selection filter  
boolstatus = swApp.GetApplySelectionFilter  
If boolstatus = False Then  
    swApp.SetApplySelectionFilter True  
Else  
    swApp.SetApplySelectionFilter False  
End If
```

```
End Sub
```

Clear Display States Example (C#)

This example shows how to clear the display states from a part. This example also shows how to remove appearances and show any hidden features.

```
//-----  
// Preconditions: Open a part document that has any of the following:  
// * appearances  
// * linked display states  
// * hidden features  
//  
// Postconditions:  
// 1. Removes appearances from all configurations of the part.  
// 2. Clears display states from all configurations.  
// 3. Shows hidden features.  
//-----  
  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace RemoveAllDisplayStates_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 modelDoc;  
        PartDoc partDoc;  
  
        bool boolstatus;  
        public void Main()  
        {  
            modelDoc = (ModelDoc2)swApp.ActiveDoc;  
            partDoc = (PartDoc)modelDoc;  
            boolstatus = partDoc.RemoveAllDisplayStates();  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Clear Display States Example (VBA)

This example shows how to clear the display states from a part. This example also shows how to remove appearances and show any hidden features.

```
'-----  
' Preconditions: Open a part document that has any of the following:  
' * appearances  
' * linked display states  
' * hidden features  
'  
' Postconditions:  
' 1. Removes appearances from all configurations of the part.  
' 2. Clears display states from all configurations.  
' 3. Shows hidden features.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim modelDoc As SldWorks.ModelDoc2  
Dim partDoc As SldWorks.PartDoc  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set modelDoc = swApp.ActiveDoc  
    Set partDoc = modelDoc  
    boolstatus = partDoc.RemoveAllDisplayStates  
End Sub
```

Clear Display States Example (VB.NET)

This example shows how to clear the display states from a part. This example also shows how to remove appearances and show any hidden features.

```
'-----  
' Preconditions: Open a part document that has any of the following:  
' * appearances  
' * linked display states  
' * hidden features  
'  
' Postconditions:  
' 1. Removes appearances from all configurations of the part.  
' 2. Clears display states from all configurations.  
' 3. Shows hidden features.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
Dim modelDoc As ModelDoc2
```

```
Dim partDoc As PartDoc
```

```
Dim boolstatus As Boolean
```

```
Sub main()
```

```
modelDoc = swApp.ActiveDoc
```

```
partDoc = modelDoc
```

```
boolstatus = partDoc.RemoveAllDisplayStates
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Close All Documents Example (VBA)

This example shows how to close all documents in a SOLIDWORKS Design session, except any modified documents.

```
'-----  
' Preconditions:  
' 1. Open two or more model documents.  
' 2. Modify the model in one model document.  
' 3. Open the Immediate window.  
' 4. Click Window in SOLIDWORKS Design to see the names of the  
'    open model documents.  
'  
' Postconditions:  
' 1. Closes all model documents, except the model document  
'    that you modified.  
' 2. Examine the Immediate window.  
' 3. Click Window again to verify that all documents,  
'    except the document that you modified, closed.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    ' If false, then all documents, excluding any dirty documents, are closed  
    boolstatus = swApp.CloseAllDocuments(False)  
    Debug.Print "All documents, excluding dirty documents, closed: " & boolstatus  
  
    ' If true, then all documents, including any dirty documents, are closed  
    'boolstatus = swApp.CloseAllDocuments(True)  
    'Debug.Print "All documents, including dirty documents, closed: " & boolstatus  
  
End Sub
```

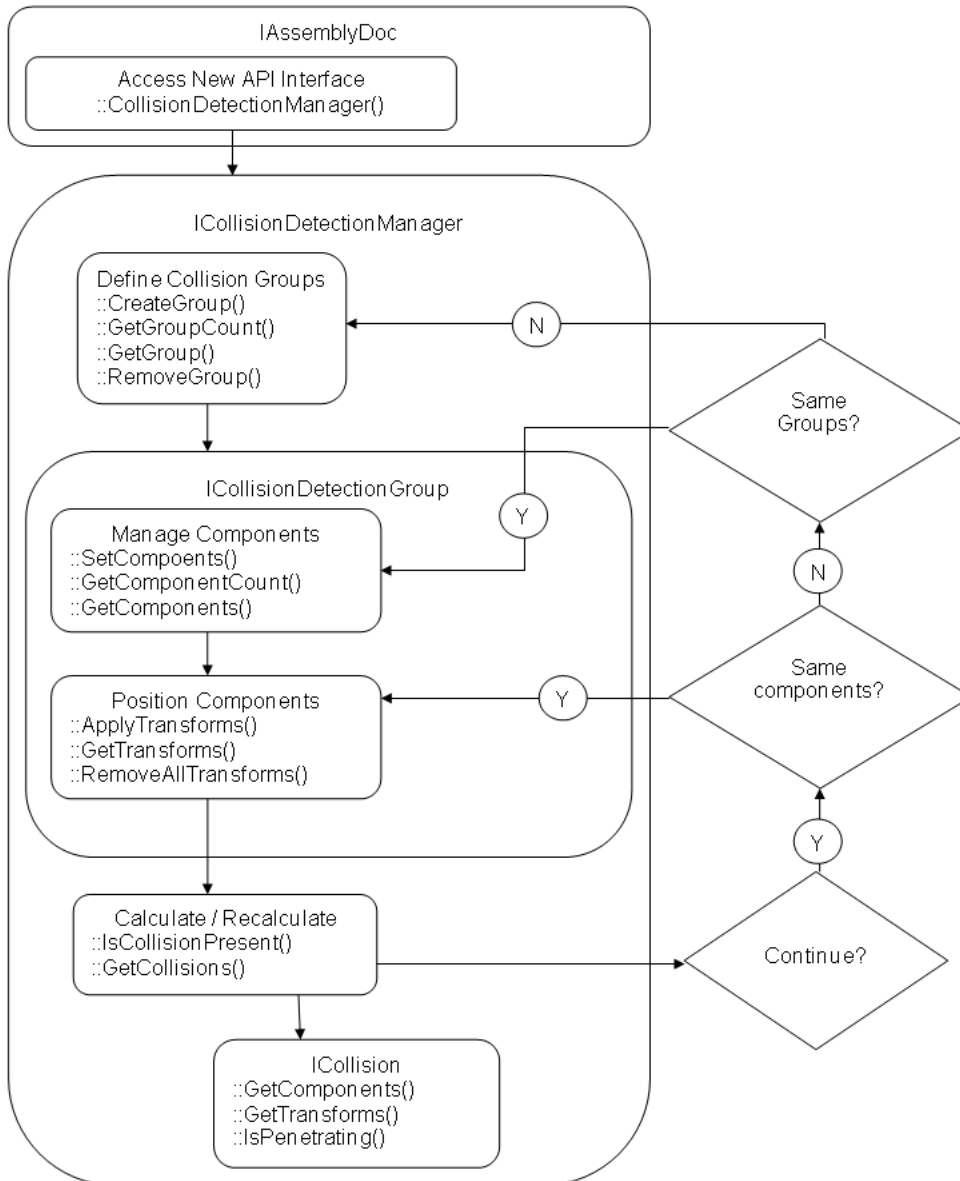
Close Document Example (VBA)

This example shows how to create new parts and close the parts by name.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens a new part document titled Part1 and gets its title.  
' 2. Opens another new part document titled Part2 and gets  
'    its title.  
' 3. Release the pointer to Part2 and closes Part2.  
' 4. Activates Part1.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part1 As SldWorks.PartDoc  
Dim Part2 As SldWorks.PartDoc  
Dim partTitle1 As String  
Dim partTitle2 As String  
Dim errors As Long  
  
Sub main()  
  
    Set swApp = CreateObject("SldWorks.Application")  
  
    ' Open a new part  
    Set Part1 = swApp.NewPart  
  
    ' Get the part's title  
    partTitle1 = Part1.GetTitle  
    Debug.Print "Part1's title: " & partTitle1  
  
    ' Open another new part  
    Set Part2 = swApp.NewPart  
  
    ' Get the part's title  
    partTitle2 = Part2.GetTitle  
    Debug.Print "Part2's title: " & partTitle2  
  
    ' Release the pointer to Part2  
    Set Part2 = Nothing  
  
    ' Close the Part2  
    swApp.CloseDoc partTitle2  
  
    ' Activate Part1  
    Set Part1 = swApp.ActivateDoc2(partTitle1, True, errors)  
    If ((errors <> 0) Or (Part1 Is Nothing)) Then  
        swApp.SendMsgToUser "Error activating Part1 document"  
    End If  
  
End Sub
```

Collision Detection Workflow

Conceptual Workflow Diagram



Combine Assembly Components into Part Example (VBA)

This example shows how to combine two assembly components into a part.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\landing_gear.sldasm.
' 2. In the FeatureManager design tree:
'   a. Click oleostrut<1>.
'   b. Ctrl+click oleopiston<1>.
'   NOTE: Both components are single-body components that
'         only contain solid bodies.
' 3. Verify that the specified part document template exists.
'
' Postconditions:
' 1. Creates a new part, which is a Boolean addition of both
'   selected components.
' 2. Click Zoom to Fit.
'
' NOTES:
' * This example:
'   * does not replace saving an assembly as a part.
'   * only illustrates the use of several geometric and
'     topological methods and properties.
' * Because the assembly is used elsewhere, do not save
'   changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp1 As SldWorks.Component2
    Dim swComp2 As SldWorks.Component2
    Dim swXform1 As SldWorks.MathTransform
    Dim swXform2 As SldWorks.MathTransform
    Dim swBody1 As SldWorks.Body2
    Dim swBody1Copy As SldWorks.Body2
    Dim swBody2 As SldWorks.Body2
    Dim swBody2Copy As SldWorks.Body2
    Dim vBodyResArr As Variant
    Dim vBodyRes As Variant
    Dim swBodyRes As SldWorks.Body2
    Dim swPartRes As SldWorks.PartDoc
    Dim swFeatRes As SldWorks.Feature
    Dim nRetVal As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swComp1 = swSelMgr.GetSelectedObjectsComponent2(1)
    Set swComp2 = swSelMgr.GetSelectedObjectsComponent2(2)
    Set swXform1 = swComp1.Transform2
    Set swXform2 = swComp2.Transform2

    Set swBody1 = swComp1.GetBody
    Set swBody2 = swComp2.GetBody
    Set swBody1Copy = swBody1.Copy
    Set swBody2Copy = swBody2.Copy

    bRet = swBody1Copy.ApplyTransform(swXform1)
    bRet = swBody2Copy.ApplyTransform(swXform2)

    vBodyResArr = swBody1Copy.Operations2(SWBODYADD, swBody2Copy, nRetVal)
    Debug.Assert Not IsEmpty(vBodyResArr)

    Set swPartRes = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2014\templates\Part.prtdot", 0, 0, 0)
    For Each vBodyRes In vBodyResArr
        Set swBodyRes = vBodyRes
        Set swFeatRes = swPartRes.CreateFeatureFromBody3(swBodyRes, False, swCreateFeatureBodyCheck + swCreateFeatureBodySimplify): Debug.Assert Not
        swFeatRes Is Nothing
    Next

End Sub

```

Combine Bodies Example (C#)

This example shows how to combine bodies in a multibody part.

```
//-----
// Preconditions:
// 1. Verify that the part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects two solid bodies.
// 3. Inserts a combine feature using the two selected
//    bodies.
// 4. Prints the type of combine feature to the Immediate
//    window.
// 5. Examine the Immediate window.
//
// NOTE: Because the part document is used elsewhere, do not
// save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CombineBodiesCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeature = default(Feature);
            CombineBodiesFeatureData swCombineBodiesFeatureData = default(CombineBodiesFeatureData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, false, 2, null, 0);
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, true, 2, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.InsertCombineFeature((int)swBodyOperationType_e.SWBODYADD, null, null);

            swCombineBodiesFeatureData = (CombineBodiesFeatureData)swFeature.GetDefinition();
            status = swCombineBodiesFeatureData.AccessSelections(swModel, null);
            //swCombineBodiesOperationType_e:
            // swCombineBodiesOperationAdd = 0
            // swCombineBodiesOperationCommon = 2
            // swCombineBodiesOperationSubtract = 1
            Debug.Print("Type of combine feature: " + swCombineBodiesFeatureData.OperationType);
            swCombineBodiesFeatureData.ReleaseSelectionAccess();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Combine Bodies Example (VBA)

This example shows how to combine bodies in a multibody part.

```
'-----
' Preconditions:
' 1. Verify that the part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects two solid bodies.
' 3. Inserts a combine feature using the two selected
'    bodies.
' 4. Prints the type of combine feature to the Immediate
'    window.
' 5. Examine the Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not
'       save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swCombineBodiesFeatureData As SldWorks.CombineBodiesFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 2, Nothing, 0)
    status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 2, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.InsertCombineFeature(swBodyOperationType_e.SWBODYADD, Nothing, Nothing)

    Set swCombineBodiesFeatureData = swFeature.GetDefinition
    status = swCombineBodiesFeatureData.AccessSelections(swModel, Nothing)
    'swCombineBodiesOperationType_e:
    ' swCombineBodiesOperationAdd = 0
    ' swCombineBodiesOperationCommon = 2
    ' swCombineBodiesOperationSubtract = 1
    Debug.Print "Type of combine feature: " & swCombineBodiesFeatureData.OperationType
    swCombineBodiesFeatureData.ReleaseSelectionAccess

End Sub
```

Combine Bodies Example (VB.NET)

This example shows how to combine bodies in a multibody part.

```

-----
Preconditions:
1. Verify that the part document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified part document.
2. Selects two solid bodies.
3. Inserts a combine feature using the two selected
   bodies.
4. Prints the type of combine feature to the Immediate
   window.
5. Examine the Immediate window.
NOTE: Because the part document is used elsewhere, do not
save changes.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureMgr As FeatureManager
        Dim swFeature As Feature
        Dim swCombineBodiesFeatureData As CombineBodiesFeatureData
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 2, Nothing, 0)
        status = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, True, 2, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.InsertCombineFeature(swBodyOperationType_e.SWBODYADD, Nothing, Nothing)

        swCombineBodiesFeatureData = swFeature.GetDefinition
        status = swCombineBodiesFeatureData.AccessSelections(swModel, Nothing)
        'swCombineBodiesOperationType_e:
        ' swCombineBodiesOperationAdd = 0
        ' swCombineBodiesOperationCommon = 2
        ' swCombineBodiesOperationSubtract = 1
        Debug.Print("Type of combine feature: " & swCombineBodiesFeatureData.OperationType)
        swCombineBodiesFeatureData.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```